```
package ocamporesidenc;

import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.event.DocumentEvent;
import javax.swing.event.DocumentListener;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.KeyEvent;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.security.spec.InvalidKeySpecException;
import java.sql.*;
import java.time.LocalDate;
import java.util.Arrays;
import java.util.Base64;
import java.util.Collections;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

public class Ocamporesidenc extends JFrame {
```

```
private static final long serialVersionUID = 1L;

private final Color Themaindontforget = new Color(0x16, 0x21, 0x3E);
private final Color Thenav1  = new Color(0x25, 0x35, 0x5C);
private final Color color1 = new Color(0x0E, 0x8B, 0x8B);
private final Color BG = new Color(0xF5, 0xF7, 0xFA);
private final Color DARK = new Color(0x1F, 0x29, 0x37);
private final Color GRAY = new Color(0x6B, 0x72, 0x80);
private final Color WHITE = Color.WHITE;
private final Color GREEN = new Color(0x10, 0xB9, 0x81);
private final Color ORANGE = new Color(0xF5, 0x9E, 0x0B);
private final Color RED = new Color(0xEF, 0x44, 0x44);
private final Color BLUE = new Color(0x3B, 0x82, 0xF6);
private final Color PURPLE = new Color(0x8B, 0x5C, 0xF6);

private static final String DB_URL = "jdbc:mysql://localhost:3306/ocamporesidenc";
private static final String DB_USER = "ocampo_app";
private static final String DB_PASSWORD = "Roypaks";

private static final double FULL_PAYMENT_AMOUNT      = 3500.00;
private static final double PARTIAL_PAYMENT_AMOUNT   = 1500.00;
private static final double MINIMAL_PAYMENT_AMOUNT   = 1000.00;
private static final double ADVANCE_3MO_DISCOUNT     = 899.00;
private static final double ADVANCE_3MO_AMOUNT       = 9601.00;

private static final String[] PAYMENT_METHODS = {
        "GCash", "PayPal", "Salmon", "Personal Pay"
};
```

```
private static final String HASH_PREFIX       = "pbkdf2";
private static final int    PBKDF2_ITERATIONS = 120_000;
private static final int    PBKDF2_KEY_BITS   = 256;
private static final int    SALT_BYTES        = 16;

private static final int    MAX_LOGIN_ATTEMPTS = 5;
private static final long   LOGIN_LOCKOUT_MS   = 60_000L;

private static final SecureRandom SECURE_RANDOM = new SecureRandom();

private static final char[] TEMP_PW_ALPHABET =
        "ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnpqrstuvwxyz23456789!@#$%"
                .toCharArray();

private static final Set<String> COUNTABLE_TABLES =
        Collections.unmodifiableSet(new HashSet<>(Arrays.asList(
                "users", "rooms", "payments", "applications", "rules")));

 private final Map<String, Integer> loginAttempts  = new HashMap<>();
private final Map<String, Long>    loginLockUntil = new HashMap<>();

private Connection connection;
private CardLayout cardLayout;
private JPanel content;
private String currentUsername;
private int currentUserId;
private String currentRole;
private int zoomPercent = 100;
```

```
private boolean isFullscreen = false;
private Rectangle windowedBounds;

private static final int LOGO_SIZE = 64;
private static final int LOGO_BASE_Y = 18;
private static final int LOGO_HOLDER_HEIGHT = 82;
private Timer logoBounceTimer;

private boolean archiveReady = false;

public Ocamporesidenc() {
    setTitle("Ocampo's Residence - Boarding House Management System");
    setSize(1300, 820);
    setMinimumSize(new Dimension(1100, 720));
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    Database01();
    installFullscreenKey();
    showLogin();
}

private void Database01() {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
        connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
        ensureArchiveTable();
    } catch (ClassNotFoundException | SQLException e) {
        JOptionPane.showMessageDialog(this,
```

```
                """
                Database connection failed.

                1. Start MySQL in XAMPP.
                2. Make sure database 'ocamporesidenc' exists.
                3. Add MySQL Connector/J to the project.

                """
                + e.getMessage(),
                "Database Error", JOptionPane.ERROR_MESSAGE);
        }
    }

    private boolean databaseReady() {
        if (connection == null) {
            JOptionPane.showMessageDialog(this,
                    "Database is not connected.",
                    "Database Error", JOptionPane.ERROR_MESSAGE);
            return false;
        }
        return true;
    }

    private void installFullscreenKey() {
        JRootPane root = getRootPane();
        InputMap im = root.getInputMap(JComponent.WHEN_IN_FOCUSED_WINDOW);
        ActionMap am = root.getActionMap();

        im.put(KeyStroke.getKeyStroke(KeyEvent.VK_F11, 0), "toggleFullscreen");
```

```
    am.put("toggleFullscreen", new AbstractAction() {
        private static final long serialVersionUID = 1L;
        @Override public void actionPerformed(ActionEvent e) {
            setFullscreen(!isFullscreen);
        }
    });

    im.put(KeyStroke.getKeyStroke(KeyEvent.VK_ESCAPE, 0), "exitFullscreen");
    am.put("exitFullscreen", new AbstractAction() {
        private static final long serialVersionUID = 1L;
        @Override public void actionPerformed(ActionEvent e) {
            if (isFullscreen) setFullscreen(false);
        }
    });
}

private void setFullscreen(boolean on) {
    if (on == isFullscreen) return;
    if (on) {
        windowedBounds = getBounds();
        dispose();
        setUndecorated(true);
        Rectangle screen = GraphicsEnvironment
                .getLocalGraphicsEnvironment()
                .getDefaultScreenDevice()
                .getDefaultConfiguration()
                .getBounds();
        setBounds(screen);
        setVisible(true);
```

```
        isFullscreen = true;
    } else {
        dispose();
        setUndecorated(false);
        if (windowedBounds != null) setBounds(windowedBounds);
        setVisible(true);
        isFullscreen = false;
    }
    revalidate();
    repaint();
}

private boolean validText(String value, int maxLength, String fieldName) {
    if (value == null || value.trim().isEmpty()) {
        JOptionPane.showMessageDialog(this, fieldName + " is required.");
        return false;
    }
    String trimmed = value.trim();
    if (trimmed.length() > maxLength) {
        JOptionPane.showMessageDialog(this,
                fieldName + " must be " + maxLength + " characters or fewer.");
        return false;
    }
    if (trimmed.chars().anyMatch(ch -> Character.isISOControl(ch))) {
        JOptionPane.showMessageDialog(this,
                fieldName + " contains invalid control characters.");
        return false;
    }
    return true;
```

```
    }

    private boolean validOptionalText(String value, int maxLength, String fieldName) {
        if (value == null || value.trim().isEmpty()) return true;
        return validText(value, maxLength, fieldName);
    }

    private boolean validEmail(String email) {
        if (email == null || email.trim().isEmpty()) return true;
        String value = email.trim();
        if (value.length() > 100 ||
!value.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$")) {
            JOptionPane.showMessageDialog(this, "Please enter a valid email address.");
            return false;
        }
        return true;
    }

    private boolean validPassword(char[] password) {
        if (password == null || password.length < 8 || password.length > 128) {
            JOptionPane.showMessageDialog(this,
                    "Password must be between 8 and 128 characters.");
            return false;
        }
        return true;
    }


    private static boolean isHashed(String stored) {
```

```
        return stored != null && stored.startsWith(HASH_PREFIX + "$");
    }

    private static String hashPassword(char[] password) {
        byte[] salt = new byte[SALT_BYTES];
        SECURE_RANDOM.nextBytes(salt);
        byte[] hash = pbkdf2(password, salt, PBKDF2_ITERATIONS, PBKDF2_KEY_BITS);
        Base64.Encoder enc = Base64.getEncoder();
        return HASH_PREFIX + "$" + PBKDF2_ITERATIONS + "$"
                + enc.encodeToString(salt) + "$" + enc.encodeToString(hash);
    }

    private static byte[] pbkdf2(char[] password, byte[] salt, int iterations, int keyBits) {
        try {
            SecretKeyFactory factory =
SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
            PBEKeySpec spec = new PBEKeySpec(password, salt, iterations, keyBits);
            try {
                return factory.generateSecret(spec).getEncoded();
            } finally {
                spec.clearPassword();
            }
        } catch (NoSuchAlgorithmException | InvalidKeySpecException e) {
            throw new IllegalStateException("Password hashing unavailable", e);
        }
    }

    /**
     * Constant-time verification. Supports legacy plain text rows so existing
```

```
     * accounts keep working; the caller re-hashes them after a successful login.
     */
    private static boolean verifyPassword(char[] password, String stored) {
        if (password == null || stored == null) return false;

        if (!isHashed(stored)) {
            byte[] a = new String(password).getBytes(StandardCharsets.UTF_8);
            byte[] b = stored.getBytes(StandardCharsets.UTF_8);
            return MessageDigest.isEqual(a, b);
        }

        String[] parts = stored.split("\\$");
        if (parts.length != 4) return false;
        try {
            int iterations = Integer.parseInt(parts[1]);
            byte[] salt = Base64.getDecoder().decode(parts[2]);
            byte[] expected = Base64.getDecoder().decode(parts[3]);
            byte[] actual = pbkdf2(password, salt, iterations, expected.length * 8);
            return MessageDigest.isEqual(expected, actual);
        } catch (RuntimeException e) {
            return false;
        }
    }

    private String generateTempPassword() {
        StringBuilder sb = new StringBuilder(14);
        for (int i = 0; i < 14; i++) {

sb.append(TEMP_PW_ALPHABET[SECURE_RANDOM.nextInt(TEMP_PW_ALPHABET.length
```

```
)]);
        }
        return sb.toString();
    }

    private String generateReferenceNo() {
        String ts = new java.text.SimpleDateFormat("yyMMddHHmmss")
                .format(new java.util.Date());
        StringBuilder sb = new StringBuilder("REF-").append(ts).append('-');
        for (int i = 0; i < 4; i++) {
            sb.append((char) ('A' + SECURE_RANDOM.nextInt(26)));
        }
        return sb.toString();
    }

    private JPanel fieldGroup(String labelText, JComponent field) {
        JPanel group = new JPanel(new BorderLayout(0, 6));
        group.setOpaque(false);
        group.setPreferredSize(new Dimension(320, 64));
        group.setMaximumSize(new Dimension(320, 64));
        group.setMinimumSize(new Dimension(320, 64));
        group.setAlignmentX(Component.CENTER_ALIGNMENT);

        JLabel l = label(labelText, true);
        group.add(l, BorderLayout.NORTH);
        group.add(field, BorderLayout.CENTER);
        return group;
    }
```

```
private void showLogin() {
    getContentPane().removeAll();
    setLayout(new GridBagLayout());
    getContentPane().setBackground(BG);

    if (logoBounceTimer != null) {
        logoBounceTimer.stop();
        logoBounceTimer = null;
    }

    JPanel card = new JPanel();
    card.setLayout(new BoxLayout(card, BoxLayout.Y_AXIS));
    card.setBackground(WHITE);
    card.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(0xE5, 0xE7, 0xEB)),
            new EmptyBorder(40, 55, 35, 55)));

    JLabel logo = new JLabel("\uD83C\uDFE0", SwingConstants.CENTER);

    logo.setFont(new Font("Segoe UI Emoji", Font.PLAIN, 44));
    logo.setForeground(color1);
    logo.setBounds(0, LOGO_BASE_Y, LOGO_SIZE, LOGO_SIZE);

    JPanel logoHolder = new JPanel(null);
    logoHolder.setOpaque(false);
    logoHolder.setAlignmentX(Component.CENTER_ALIGNMENT);
    logoHolder.setPreferredSize(new Dimension(LOGO_SIZE, LOGO_HOLDER_HEIGHT));
    logoHolder.setMaximumSize(new Dimension(LOGO_SIZE, LOGO_HOLDER_HEIGHT));
    logoHolder.setMinimumSize(new Dimension(LOGO_SIZE, LOGO_HOLDER_HEIGHT));
```

```
        logoHolder.add(logo);
        installLogoBounce(logo, logoHolder);

        JLabel title = new JLabel("Ocampo's Residence", SwingConstants.CENTER);
        title.setFont(new Font("Segoe UI", Font.BOLD, 22));
        title.setForeground(DARK);
        title.setAlignmentX(Component.CENTER_ALIGNMENT);

        JLabel sub = new JLabel("Boarding House Management System",
SwingConstants.CENTER);
        sub.setFont(new Font("Segoe UI", Font.PLAIN, 11));
        sub.setForeground(GRAY);
        sub.setAlignmentX(Component.CENTER_ALIGNMENT);

        JTextField username = new JTextField();
        styleInput(username);

        JPasswordField password = new JPasswordField();
        styleInput(password);

        JButton login = button("Sign In", color1);
        login.setPreferredSize(new Dimension(320, 42));
        login.setMaximumSize(new Dimension(320, 42));
        login.setMinimumSize(new Dimension(320, 42));
        login.setAlignmentX(Component.CENTER_ALIGNMENT);
        login.addActionListener(e -> {
            char[] pw = password.getPassword();
            try {
                loginUser(username.getText().trim(), pw);
```

```
    } finally {
        java.util.Arrays.fill(pw, '\0');
    }
});

username.addActionListener(e -> password.requestFocusInWindow());
password.addActionListener(e -> login.doClick());

JLabel forgot = new JLabel("<html><a href='#' style='color:#0E8B8B;'>"
        + "Forgot Password? Click here.</a></html>", SwingConstants.CENTER);
forgot.setFont(new Font("Segoe UI", Font.PLAIN, 11));
forgot.setForeground(color1);
forgot.setAlignmentX(Component.CENTER_ALIGNMENT);
forgot.setCursor(new Cursor(Cursor.HAND_CURSOR));
forgot.addMouseListener(new java.awt.event.MouseAdapter() {
    @Override
    public void mouseClicked(java.awt.event.MouseEvent e) {
        showForgotPasswordDialog();
    }
});

card.add(logoHolder);
card.add(Box.createVerticalStrut(4));
card.add(title);
card.add(Box.createVerticalStrut(5));
card.add(sub);
card.add(Box.createVerticalStrut(28));
card.add(fieldGroup("Username", username));
card.add(Box.createVerticalStrut(12));
```

```
    card.add(fieldGroup("Password", password));
    card.add(Box.createVerticalStrut(22));
    card.add(login);
    card.add(Box.createVerticalStrut(18));
    card.add(forgot);

    add(card);
    revalidate();
    repaint();
    username.requestFocusInWindow();
}


private void installLogoBounce(JLabel logo, JPanel holder) {
    logo.setCursor(new Cursor(Cursor.HAND_CURSOR));
    holder.setCursor(new Cursor(Cursor.HAND_CURSOR));

    java.awt.event.MouseAdapter trigger = new java.awt.event.MouseAdapter() {
        @Override
        public void mouseEntered(java.awt.event.MouseEvent e) { bounceLogo(logo); }

        @Override
        public void mouseClicked(java.awt.event.MouseEvent e) { bounceLogo(logo); }
    };

    logo.addMouseListener(trigger);
    holder.addMouseListener(trigger);
}
```

```
private void bounceLogo(JLabel logo) {
    if (logoBounceTimer != null && logoBounceTimer.isRunning()) return;

    final int totalSteps = 32;
    final int amplitude = 16;
    final int[] step = {0};

    logoBounceTimer = new Timer(16, null);
    logoBounceTimer.addActionListener(e -> {
        step[0]++;
        double progress = (double) step[0] / totalSteps;
        double wave = Math.abs(Math.sin(progress * Math.PI * 3));
        double decay = 1.0 - progress;
        int offset = (int) Math.round(wave * amplitude * decay);

        logo.setLocation(0, LOGO_BASE_Y - offset);

        if (step[0] >= totalSteps) {
            logoBounceTimer.stop();
            logo.setLocation(0, LOGO_BASE_Y);
        }
    });
    logoBounceTimer.start();
}

private void loginUser(String username, char[] password) {
    if (!databaseReady()) return;
    if (username == null || username.isEmpty() || username.length() > 50 ||
            password == null || password.length == 0 || password.length > 128) {
```

```
    JOptionPane.showMessageDialog(this, "Enter a valid username and password.",
            "Login Failed", JOptionPane.WARNING_MESSAGE);
    return;
}

Long lockUntil = loginLockUntil.get(username);
if (lockUntil != null && System.currentTimeMillis() < lockUntil) {
    long secs = (lockUntil - System.currentTimeMillis()) / 1000 + 1;
    JOptionPane.showMessageDialog(this,
            "Too many failed attempts.\nTry again in " + secs + " second(s).",
            "Login Locked", JOptionPane.WARNING_MESSAGE);
    return;
}

String sql = "SELECT user_id, username, password, role FROM users "
        + "WHERE username=? AND status='Active'";

try (PreparedStatement ps = connection.prepareStatement(sql)) {
    ps.setString(1, username);

    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next() && verifyPassword(password, rs.getString("password"))) {
            String stored = rs.getString("password");
            currentUserId = rs.getInt("user_id");
            currentUsername = rs.getString("username");
            currentRole = rs.getString("role");

            loginAttempts.remove(username);
            loginLockUntil.remove(username);
```

```
            if (!isHashed(stored)) {
                try (PreparedStatement up = connection.prepareStatement(
                        "UPDATE users SET password=? WHERE user_id=?")) {
                    up.setString(1, hashPassword(password));
                    up.setInt(2, currentUserId);
                    up.executeUpdate();
                } catch (SQLException ignored) { /* non-fatal */ }
            }

            if ("ADMIN".equalsIgnoreCase(currentRole)) showAdmin();
            else showTenant();
            return;
        }

        registerFailedAttempt(username);
        JOptionPane.showMessageDialog(this,
                "Invalid username/password or account is inactive.",
                "Login Failed", JOptionPane.WARNING_MESSAGE);
      }
   } catch (SQLException e) {
      showError(e);
   }
}

private void registerFailedAttempt(String username) {
   int attempts = loginAttempts.getOrDefault(username, 0) + 1;
   if (attempts >= MAX_LOGIN_ATTEMPTS) {
```

```
            loginLockUntil.put(username, System.currentTimeMillis() + LOGIN_LOCKOUT_MS);
            loginAttempts.put(username, 0);
        } else {
            loginAttempts.put(username, attempts);
        }
    }


    private void showForgotPasswordDialog() {
        if (!databaseReady()) return;

        JComboBox<String> roleBox = new JComboBox<>(new String[]{"TENANT", "ADMIN"});
        JTextField nameField = new JTextField();
        JTextField contactField = new JTextField();

        JPanel panel = new JPanel(new GridBagLayout());
        GridBagConstraints g = new GridBagConstraints();
        g.insets = new Insets(6, 6, 6, 6);
        g.fill = GridBagConstraints.HORIZONTAL;

        g.gridx = 0; g.gridy = 0; g.weightx = 0;
        panel.add(new JLabel("Account Type:"), g);
        g.gridx = 1; g.weightx = 1;
        panel.add(roleBox, g);

        g.gridx = 0; g.gridy = 1; g.weightx = 0;
        panel.add(new JLabel("Full Name or Username:"), g);
        g.gridx = 1; g.weightx = 1;
        panel.add(nameField, g);
```

```
g.gridx = 0; g.gridy = 2; g.weightx = 0;
panel.add(new JLabel("Email or Phone:"), g);
g.gridx = 1; g.weightx = 1;
panel.add(contactField, g);

int result = JOptionPane.showConfirmDialog(this, panel,
        "Forgot Password", JOptionPane.OK_CANCEL_OPTION,
        JOptionPane.PLAIN_MESSAGE);
if (result != JOptionPane.OK_OPTION) return;

String name = nameField.getText().trim();
String contact = contactField.getText().trim();
String role = roleBox.getSelectedItem().toString();

if (name.isEmpty() || contact.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Please fill in all fields.");
    return;
}

String sql = "SELECT user_id, username FROM users WHERE role=? "
        + "AND (full_name=? OR username=?) "
        + "AND (email=? OR phone=?)";

try (PreparedStatement ps = connection.prepareStatement(sql)) {
    ps.setString(1, role);
    ps.setString(2, name);
    ps.setString(3, name);
    ps.setString(4, contact);
```

```
    ps.setString(5, contact);

    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
            String tempPw = generateTempPassword();

            try (PreparedStatement up = connection.prepareStatement(
                    "UPDATE users SET password=? WHERE user_id=?")) {
                up.setString(1, hashPassword(tempPw.toCharArray()));
                up.setInt(2, rs.getInt("user_id"));
                up.executeUpdate();
            }

            JOptionPane.showMessageDialog(this,
                    """
                    Password reset request verified.

                    Your temporary password is:  """ + tempPw + "\n\n"
                    + "Please log in and change it immediately.",
                    "Password Reset", JOptionPane.INFORMATION_MESSAGE);
        } else {
            JOptionPane.showMessageDialog(this, """
                                        No matching account found.
                                        Please check your details and try again.""",
                    "Not Found", JOptionPane.WARNING_MESSAGE);
        }
    }
} catch (SQLException e) {
    showError(e);
```

```
    }
}


private JLabel label(String text, boolean bold) {
    JLabel l = new JLabel(text);
    l.setFont(new Font("Segoe UI", bold ? Font.BOLD : Font.PLAIN, 11));
    l.setForeground(DARK);
    return l;
}

private void styleInput(JTextField field) {
    field.setMaximumSize(new Dimension(320, 36));
    field.setPreferredSize(new Dimension(320, 36));
    field.setMinimumSize(new Dimension(320, 36));
    field.setFont(new Font("Segoe UI", Font.PLAIN, 12));
    field.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(0xE5, 0xE7, 0xEB)),
            new EmptyBorder(6, 12, 6, 12)));
}

private JButton button(String text, Color bg) {
    JButton b = new JButton(text);
    b.setBackground(bg);
    b.setForeground(WHITE);
    b.setFocusPainted(false);
    b.setBorderPainted(false);
    b.setFont(new Font("Segoe UI", Font.BOLD, 12));
    b.setCursor(new Cursor(Cursor.HAND_CURSOR));
```

```
    return b;
}

private JPanel whiteCard() {
    JPanel p = new JPanel();
    p.setBackground(WHITE);
    p.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(0xE5, 0xE7, 0xEB)),
            new EmptyBorder(16, 16, 16, 16)));
    return p;
}

private JPanel header(String title, String subtitle) {
    JPanel h = new JPanel(new BorderLayout());
    h.setOpaque(false);
    h.setBorder(new EmptyBorder(0, 0, 18, 0));

    JPanel text = new JPanel();
    text.setOpaque(false);
    text.setLayout(new BoxLayout(text, BoxLayout.Y_AXIS));

    JLabel t = new JLabel(title);
    t.setFont(new Font("Segoe UI", Font.BOLD, 20));
    t.setForeground(DARK);

    JLabel s = new JLabel(subtitle);
    s.setFont(new Font("Segoe UI", Font.PLAIN, 11));
    s.setForeground(GRAY);
```

```
        text.add(t);
        text.add(Box.createVerticalStrut(4));
        text.add(s);

        JLabel profile = new JLabel("\uD83D\uDC64 " + currentUsername + "  \u2022  " +
currentRole);
        profile.setFont(new Font("Segoe UI", Font.BOLD, 12));
        profile.setForeground(DARK);

        JButton search = new JButton("Search");
        search.setFont(new Font("Segoe UI", Font.BOLD, 11));
        search.setForeground(DARK);
        search.setBackground(WHITE);
        search.setFocusPainted(false);
        search.setBorder(BorderFactory.createCompoundBorder(
                BorderFactory.createLineBorder(new Color(0xD1, 0xD5, 0xDB)),
                new EmptyBorder(4, 12, 4, 12)));
        search.setCursor(new Cursor(Cursor.HAND_CURSOR));
        search.setToolTipText("Search pages and features");
        search.addActionListener(e -> openSearchDialog());

        JButton zoomOut = new JButton("\u2212");
        JButton zoomIn = new JButton("+");
        styleZoomButton(zoomOut);
        styleZoomButton(zoomIn);
        zoomOut.setToolTipText("Zoom out");
        zoomIn.setToolTipText("Zoom in");
        zoomOut.addActionListener(e -> changeZoom(-10));
        zoomIn.addActionListener(e -> changeZoom(10));
```

```
    JPanel right = new JPanel(new FlowLayout(FlowLayout.RIGHT, 6, 0));
    right.setOpaque(false);
    right.add(profile);
    right.add(Box.createHorizontalStrut(8));
    right.add(search);
    right.add(zoomOut);
    right.add(zoomIn);

    h.add(text, BorderLayout.WEST);
    h.add(right, BorderLayout.EAST);
    return h;
}


private void openSearchDialog() {
    JTextField query = new JTextField();
    Object[] fields = {
            "What are you looking for?",
            "Try: payments, room, profile, rules, recovery, applications, archive...",
            query
    };
    int result = JOptionPane.showConfirmDialog(this, fields,
            "Search", JOptionPane.OK_CANCEL_OPTION);
    if (result != JOptionPane.OK_OPTION) return;
    navigateTo(query.getText().trim());
}

private void navigateTo(String query) {
```

```
if (query == null || query.isEmpty()) return;
if (cardLayout == null || content == null) {
    JOptionPane.showMessageDialog(this, "Please log in first.");
    return;
}

String q = query.toLowerCase();
boolean admin = "ADMIN".equalsIgnoreCase(currentRole);
String page = null;

if (q.contains("dash") || q.contains("home") || q.contains("overview")) {
    page = "Dashboard";
} else if (q.contains("archiv") || q.contains("history") || q.contains("deleted")) {
    page = admin ? "Archive" : null;
} else if (q.contains("pay") || q.contains("invoice") || q.contains("bill")) {
    page = "Payments";
} else if (q.contains("profil") || q.contains("account") || q.contains("me")) {
    page = admin ? null : "Profile";
} else if (q.contains("user") || q.contains("tenant") || q.contains("account list")) {
    page = admin ? "Users" : null;
} else if (q.contains("room") || q.contains("slot") || q.contains("bed")) {
    page = admin ? "Rooms" : "Application";
} else if (q.contains("appl") || q.contains("request") || q.contains("apply")) {
    page = admin ? "Applications" : "Application";
} else if (q.contains("rule") || q.contains("polic") || q.contains("guide")) {
    page = "Rules";
} else if (q.contains("recover") || q.contains("forgot")
        || q.contains("reset") || q.contains("password")) {
    page = admin ? "Recovery" : "Profile";
```

```
    }

    if (page == null) {
        JOptionPane.showMessageDialog(this,
                "No matching page found for: \"" + query + "\"\n\n"
                + (admin
                ? "Try: dashboard, payments, users, rooms, applications, rules, recovery, archive"
                : "Try: dashboard, profile, rules, payments, application"),
                "Search", JOptionPane.INFORMATION_MESSAGE);
        return;
    }

    cardLayout.show(content, page);
}

private void styleZoomButton(JButton button) {
    button.setFont(new Font("Segoe UI", Font.BOLD, 17));
    button.setForeground(DARK);
    button.setBackground(WHITE);
    button.setFocusPainted(false);
    button.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(0xD1, 0xD5, 0xDB)),
            new EmptyBorder(2, 9, 2, 9)));
    button.setPreferredSize(new Dimension(36, 32));
    button.setCursor(new Cursor(Cursor.HAND_CURSOR));
}

private void changeZoom(int change) {
    int newZoom = Math.max(80, Math.min(150, zoomPercent + change));
```

```
    if (newZoom == zoomPercent) return;
    zoomPercent = newZoom;
    if (content != null) {
        applyZoomToTree(content);
        content.revalidate();
        content.repaint();
    }
    revalidate();
    repaint();
}

private void applyZoomToTree(Component component) {
    applyZoom(component);
    if (component instanceof Container) {
        for (Component child : ((Container) component).getComponents()) {
            applyZoomToTree(child);
        }
    }
}

private void applyZoom(Component component) {
    if (component instanceof JComponent) {
        JComponent jc = (JComponent) component;
        captureBase(jc, "basePref", true);
        captureBase(jc, "baseMax", false);
    }

    Font baseFont = null;
    if (component instanceof JComponent) {
```

```
            Object stored = ((JComponent) component).getClientProperty("baseFont");
            if (stored instanceof Font) baseFont = (Font) stored;
        }
        if (baseFont == null && component.getFont() != null) {
            baseFont = component.getFont();
            if (component instanceof JComponent) {
                ((JComponent) component).putClientProperty("baseFont", baseFont);
            }
        }
        if (baseFont != null) {
            float size = Math.max(8f, baseFont.getSize2D() * zoomPercent / 100f);
            component.setFont(baseFont.deriveFont(size));
        }

        if (component instanceof JComponent) {
            JComponent jc = (JComponent) component;
            applyScaled(jc, "basePref", true);
            applyScaled(jc, "baseMax", false);
        }

        if (component instanceof JTable) {
            JTable table = (JTable) component;
            table.setRowHeight(Math.max(20, Math.round(34 * zoomPercent / 100f)));
        }
    }

    private void captureBase(JComponent jc, String key, boolean isPref) {
        if (jc.getClientProperty(key) != null) return;
        Dimension current = isPref ? jc.getPreferredSize() : jc.getMaximumSize();
```

```
    if (current == null) return;
    boolean finite = current.width < Integer.MAX_VALUE
            || current.height < Integer.MAX_VALUE;
    if (!finite) return;
    jc.putClientProperty(key, new Dimension(current));
}

private void applyScaled(JComponent jc, String key, boolean isPref) {
    Dimension base = (Dimension) jc.getClientProperty(key);
    if (base == null) return;
    int w = base.width == Integer.MAX_VALUE ? Integer.MAX_VALUE
            : Math.max(0, Math.round(base.width * zoomPercent / 100f));
    int h = base.height == Integer.MAX_VALUE ? Integer.MAX_VALUE
            : Math.max(0, Math.round(base.height * zoomPercent / 100f));
    if (isPref) jc.setPreferredSize(new Dimension(w, h));
    else jc.setMaximumSize(new Dimension(w, h));
}

private JPanel basePanel(String title, String subtitle) {
    JPanel p = new JPanel(new BorderLayout(0, 15));
    p.setBackground(BG);
    p.setBorder(new EmptyBorder(22, 26, 22, 26));
    p.add(header(title, subtitle), BorderLayout.NORTH);
    return p;
}

private void setPageTop(JPanel page, JComponent additionalTop) {
    Component existingHeader = ((BorderLayout) page.getLayout())
            .getLayoutComponent(page, BorderLayout.NORTH);
```

```
    JPanel topStack = new JPanel();
    topStack.setOpaque(false);
    topStack.setLayout(new BoxLayout(topStack, BoxLayout.Y_AXIS));

    if (existingHeader != null) {
        page.remove(existingHeader);
        topStack.add(existingHeader);
    }
    topStack.add(additionalTop);
    page.add(topStack, BorderLayout.NORTH);
}

private void styleTable(JTable table) {
    table.setRowHeight(34);
    table.setFont(new Font("Segoe UI", Font.PLAIN, 12));
    table.getTableHeader().setFont(new Font("Segoe UI", Font.BOLD, 11));
    table.getTableHeader().setBackground(new Color(0xF0, 0xF0, 0xF0));
    table.setGridColor(new Color(0xE5, 0xE7, 0xEB));
    table.setShowVerticalLines(false);
    table.setFillsViewportHeight(true);
    table.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

    DefaultTableCellRenderer center = new DefaultTableCellRenderer();
    center.setHorizontalAlignment(SwingConstants.CENTER);
    table.getColumnModel().getColumn(0).setCellRenderer(center);
}

private boolean confirm(String message) {
```

```
    return JOptionPane.showConfirmDialog(this, message,
            "Confirm", JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION;
}

private void rollback() {
    try {
        connection.rollback();
        connection.setAutoCommit(true);
    } catch (SQLException ignored) {}
}

private void showError(Component parent, Exception e) {
    JOptionPane.showMessageDialog(parent, e.getMessage(),
            "Error", JOptionPane.ERROR_MESSAGE);
}

private void showError(Exception e) {
    showError(this, e);
}



private void showAdmin() {
    getContentPane().removeAll();
    setLayout(new BorderLayout());

    JPanel sidebar = createSidebar();
    content = new JPanel(new CardLayout());
    content.setBackground(BG);
    cardLayout = (CardLayout) content.getLayout();
```

```
    content.add(adminDashboard(), "Dashboard");
    content.add(adminPayments(), "Payments");
    content.add(adminUsers(), "Users");
    content.add(adminRooms(), "Rooms");
    content.add(adminApplications(), "Applications");
    content.add(rulesPanel(true), "Rules");
    content.add(adminArchive(), "Archive");
    content.add(adminRecovery(), "Recovery");

    add(sidebar, BorderLayout.WEST);
    add(content, BorderLayout.CENTER);
    cardLayout.show(content, "Dashboard");

    if (content != null) applyZoomToTree(content);
    revalidate();
    repaint();
}

private JPanel createSidebar() {
    JPanel sidebar = new JPanel();
    sidebar.setBackground(Themaindontforget);
    sidebar.setPreferredSize(new Dimension(235, 780));
    sidebar.setLayout(new BoxLayout(sidebar, BoxLayout.Y_AXIS));
    sidebar.setBorder(new EmptyBorder(20, 12, 20, 12));

    JLabel brand = new JLabel("Ocampo's Residence");
    brand.setFont(new Font("Segoe UI", Font.BOLD, 16));
    brand.setForeground(WHITE);
```

```
    brand.setBorder(new EmptyBorder(0, 8, 28, 0));
    sidebar.add(brand);

    addNav(sidebar, "Dashboard", "Dashboard");
    addNav(sidebar, "Payments", "Payments");
    addNav(sidebar, "Tenants / Users", "Users");
    addNav(sidebar, "Rooms", "Rooms");
    addNav(sidebar, "Applications", "Applications");
    addNav(sidebar, "House Rules", "Rules");
    addNav(sidebar, "Transaction Archive", "Archive");
    addNav(sidebar, "Account Recovery", "Recovery");

    sidebar.add(Box.createVerticalGlue());

    JButton logout = new JButton("Logout");
    logout.setForeground(RED);
    logout.setBackground(Themaindontforget);
    logout.setContentAreaFilled(false);
    logout.setBorderPainted(false);
    logout.setFocusPainted(false);
    logout.setAlignmentX(Component.LEFT_ALIGNMENT);
    logout.setCursor(new Cursor(Cursor.HAND_CURSOR));
    logout.addActionListener(e -> logout());
    sidebar.add(logout);

    return sidebar;
}

private void installNavHover(JButton b) {
```

```
    b.setCursor(new Cursor(Cursor.HAND_CURSOR));
    b.addMouseListener(new java.awt.event.MouseAdapter() {
        @Override
        public void mouseEntered(java.awt.event.MouseEvent e) {
            boolean active = Boolean.TRUE.equals(b.getClientProperty("navActive"));
            b.setBackground(active ? color1.brighter() : Thenav1);
            b.setForeground(WHITE);
        }

        @Override
        public void mouseExited(java.awt.event.MouseEvent e) {
            boolean active = Boolean.TRUE.equals(b.getClientProperty("navActive"));
            b.setBackground(active ? color1: Themaindontforget);
            b.setForeground(WHITE);
        }
    });
}

private void selectNavButton(JPanel sidebar, JButton selected) {
    for (Component c : sidebar.getComponents()) {
        if (c instanceof JButton) {
            JButton other = (JButton) c;
            if ("Logout".equals(other.getText())) continue;
            other.putClientProperty("navActive", Boolean.FALSE);
            other.setBackground(Themaindontforget);
            other.setForeground(WHITE);
        }
    }
    selected.putClientProperty("navActive", Boolean.TRUE);
```

```
    selected.setBackground(color1);
    selected.setForeground(WHITE);
}

private void addNav(JPanel sidebar, String text, String page) {
    JButton b = new JButton(text);
    b.setMaximumSize(new Dimension(215, 38));
    b.setForeground(WHITE);
    b.setBackground(Themaindontforget);
    b.setFocusPainted(false);
    b.setBorderPainted(false);
    b.setOpaque(true);
    b.setHorizontalAlignment(SwingConstants.LEFT);
    b.setFont(new Font("Segoe UI", Font.PLAIN, 13));
    b.setBorder(new EmptyBorder(5, 12, 5, 10));

    b.addActionListener(e -> {
        cardLayout.show(content, page);
        selectNavButton(sidebar, b);
    });

    installNavHover(b);

    sidebar.add(b);
    sidebar.add(Box.createVerticalStrut(4));
}

/** Clears session state on logout. */
private void logout() {
```

```
    currentUserId = 0;
    currentUsername = null;
    currentRole = null;
    if (content != null) {
        content.removeAll();
        content = null;
    }
    cardLayout = null;
    loginAttempts.clear();
    loginLockUntil.clear();
    showLogin();
}


private JPanel adminDashboard() {
    JPanel p = basePanel("Dashboard Overview", "Boarding House Management Console");

    JScrollPane scroll = new JScrollPane(buildAdminDashboardContent());
    scroll.setBorder(null);
    scroll.getVerticalScrollBar().setUnitIncrement(16);
    scroll.setOpaque(false);
    scroll.getViewport().setOpaque(false);

    p.add(scroll, BorderLayout.CENTER);
    return p;
}

private JPanel buildAdminDashboardContent() {
    JPanel main = new JPanel();
```

```
main.setLayout(new BoxLayout(main, BoxLayout.Y_AXIS));
main.setOpaque(false);

JPanel metrics = new JPanel(new GridLayout(1, 6, 12, 0));
metrics.setOpaque(false);
metrics.setPreferredSize(new Dimension(0, 105));
metrics.setMaximumSize(new Dimension(Integer.MAX_VALUE, 105));

metrics.add(metric("TENANTS", count("users", "role='TENANT'"),
        "Registered", "\uD83D\uDC65", color1));
metrics.add(metric("AVAILABLE", availableSlots(),
        "Slots", "\uD83D\uDECF", BLUE));
metrics.add(metric("OCCUPANCY", occupancyRate(),
        "Rate", "\uD83D\uDCCA", PURPLE));
metrics.add(metric("PENDING PAY", count("payments", "status='Pending'"),
        "For review", "\u20B1", ORANGE));
metrics.add(metric("APPLICATIONS", count("applications", "status='Pending'"),
        "Pending", "\uD83D\uDCDD", BLUE));
metrics.add(metric("MONTHLY REV", monthlyRevenue(),
        "Collected", "\uD83D\uDCB5", GREEN));

main.add(metrics);
main.add(Box.createVerticalStrut(15));

JPanel roomsCard = whiteCard();
roomsCard.setLayout(new BorderLayout(0, 10));
roomsCard.setMaximumSize(new Dimension(Integer.MAX_VALUE, 240));
roomsCard.add(sectionTitle("Room Occupancy Summary"), BorderLayout.NORTH);
```

```
String[] cols = {"Room", "Capacity", "Occupied", "Available", "Rent", "Status"};
DefaultTableModel model = new DefaultTableModel(cols, 0) {
    @Override
    public boolean isCellEditable(int r, int c) { return false; }
};

String sql = "SELECT room_number, capacity, occupied, "
        + "(capacity-occupied) available, monthly_rent FROM rooms ORDER BY room_id";

try (PreparedStatement ps = connection.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        int occupied = rs.getInt("occupied");
        int capacity = rs.getInt("capacity");
        model.addRow(new Object[]{
                rs.getString("room_number"),
                capacity,
                occupied,
                rs.getInt("available"),
                "\u20B1" + rs.getBigDecimal("monthly_rent"),
                occupied >= capacity ? "FULL" : "AVAILABLE"
        });
    }
} catch (SQLException e) {
    showError(e);
}

JTable table = new JTable(model);
styleTable(table);
```

```
table.getColumnModel().getColumn(5).setCellRenderer(new StatusRenderer());
JScrollPane sp = new JScrollPane(table);
sp.setPreferredSize(new Dimension(0, 180));
roomsCard.add(sp, BorderLayout.CENTER);
main.add(roomsCard);
main.add(Box.createVerticalStrut(15));

JPanel row = new JPanel(new GridLayout(1, 2, 15, 0));
row.setOpaque(false);
row.setMaximumSize(new Dimension(Integer.MAX_VALUE, 260));

row.add(recentPaymentsAdminCard());
row.add(recentApplicationsAdminCard());

main.add(row);
main.add(Box.createVerticalStrut(15));

JPanel revCard = whiteCard();
revCard.setLayout(new BorderLayout(0, 10));
revCard.setMaximumSize(new Dimension(Integer.MAX_VALUE, 160));
revCard.add(sectionTitle("Revenue Breakdown"), BorderLayout.NORTH);

JPanel revGrid = new JPanel(new GridLayout(1, 3, 15, 0));
revGrid.setOpaque(false);

revGrid.add(revenueTile("Collected (Paid)", totalPaidAmount(), GREEN));
revGrid.add(revenueTile("Pending Payments", totalPendingAmount(), ORANGE));
revGrid.add(revenueTile("Projected Monthly", projectedMonthly(), BLUE));
```

```
    revCard.add(revGrid, BorderLayout.CENTER);
    main.add(revCard);

    return main;
}

private JPanel metric(String title, String value, String sub, String icon, Color valueColor) {
    JPanel card = whiteCard();
    card.setLayout(new BorderLayout(0, 5));

    JLabel top = new JLabel(title + "    " + icon);
    top.setFont(new Font("Segoe UI", Font.BOLD, 10));
    top.setForeground(GRAY);

    JLabel number = new JLabel(value);
    number.setFont(new Font("Segoe UI", Font.BOLD, 22));
    number.setForeground(valueColor);

    JLabel small = new JLabel(sub);
    small.setFont(new Font("Segoe UI", Font.PLAIN, 10));
    small.setForeground(GRAY);

    card.add(top, BorderLayout.NORTH);
    card.add(number, BorderLayout.CENTER);
    card.add(small, BorderLayout.SOUTH);
    return card;
}

private JLabel sectionTitle(String text) {
```

```
    JLabel l = new JLabel(text);
    l.setFont(new Font("Segoe UI", Font.BOLD, 13));
    l.setForeground(DARK);
    return l;
}

private JPanel recentPaymentsAdminCard() {
    JPanel panel = whiteCard();
    panel.setLayout(new BorderLayout(0, 8));
    panel.add(sectionTitle("Recent Payments"), BorderLayout.NORTH);

    String[] cols = {"Tenant", "Amount", "Status"};
    DefaultTableModel model = new DefaultTableModel(cols, 0) {
        @Override
        public boolean isCellEditable(int r, int c) { return false; }
    };

    String sql = "SELECT u.username, p.amount, p.status FROM payments p "
            + "JOIN users u ON p.user_id=u.user_id "
            + "ORDER BY p.payment_id DESC LIMIT 8";

    try (PreparedStatement ps = connection.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            model.addRow(new Object[]{
                    rs.getString("username"),
                    "\u20B1" + rs.getBigDecimal("amount"),
                    rs.getString("status")
            });
```

```
        }
    } catch (SQLException e) {
        showError(e);
    }

    JTable table = new JTable(model);
    styleTable(table);
    table.getColumnModel().getColumn(2).setCellRenderer(new StatusRenderer());
    panel.add(new JScrollPane(table), BorderLayout.CENTER);
    return panel;
}

private JPanel recentApplicationsAdminCard() {
    JPanel panel = whiteCard();
    panel.setLayout(new BorderLayout(0, 8));
    panel.add(sectionTitle("Recent Applications"), BorderLayout.NORTH);

    String[] cols = {"Tenant", "Room", "Status"};
    DefaultTableModel model = new DefaultTableModel(cols, 0) {
        @Override
        public boolean isCellEditable(int r, int c) { return false; }
    };

    String sql = "SELECT u.username, r.room_number, a.status FROM applications a "
            + "JOIN users u ON a.user_id=u.user_id "
            + "JOIN rooms r ON a.room_id=r.room_id "
            + "ORDER BY a.application_id DESC LIMIT 8";

    try (PreparedStatement ps = connection.prepareStatement(sql);
```

```
         ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            model.addRow(new Object[]{
                    rs.getString("username"),
                    rs.getString("room_number"),
                    rs.getString("status")
            });
        }
    } catch (SQLException e) {
        showError(e);
    }

    JTable table = new JTable(model);
    styleTable(table);
    table.getColumnModel().getColumn(2).setCellRenderer(new StatusRenderer());
    panel.add(new JScrollPane(table), BorderLayout.CENTER);
    return panel;
}

private JPanel revenueTile(String label, String value, Color color) {
    JPanel tile = new JPanel(new BorderLayout(0, 5));
    tile.setBackground(WHITE);
    tile.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(0xE5, 0xE7, 0xEB)),
            new EmptyBorder(12, 14, 12, 14)));

    JLabel l = new JLabel(label);
    l.setFont(new Font("Segoe UI", Font.PLAIN, 11));
    l.setForeground(GRAY);
```

```
    JLabel v = new JLabel(value);
    v.setFont(new Font("Segoe UI", Font.BOLD, 20));
    v.setForeground(color);

    tile.add(l, BorderLayout.NORTH);
    tile.add(v, BorderLayout.CENTER);
    return tile;
}

private JPanel adminPayments() {
    JPanel p = basePanel("Payment Management",
            "Review, approve, reject and archive tenant payments.");

    JPanel top = new JPanel(new BorderLayout(10, 10));
    top.setOpaque(false);

    JPanel summary = new JPanel(new GridLayout(1, 4, 12, 0));
    summary.setOpaque(false);
    summary.setPreferredSize(new Dimension(0, 90));

    summary.add(metric("TOTAL RECORDS", count("payments", null),
            "All time", "\uD83D\uDCC4", color1));
    summary.add(metric("TOTAL PAID", totalPaidAmount(),
            "Collected", "\u2713", GREEN));
    summary.add(metric("PENDING", totalPendingAmount(),
            "Awaiting review", "\u23F3", ORANGE));
    summary.add(metric("REJECTED", count("payments", "status='Rejected'"),
            "Not accepted", "\u2717", RED));
```

```
top.add(summary, BorderLayout.NORTH);

JPanel filterRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 5));
filterRow.setOpaque(false);
filterRow.add(label("Filter by status:", true));
JComboBox<String> filter = new JComboBox<>(
        new String[]{"All", "Pending", "Paid", "Rejected"});
filterRow.add(filter);
top.add(filterRow, BorderLayout.SOUTH);

setPageTop(p, top);

String[] columns = {"ID", "Date", "Tenant", "Room", "Description",
        "Amount", "Method", "Reference", "Status"};
DefaultTableModel model = new DefaultTableModel(columns, 0) {
    @Override
    public boolean isCellEditable(int r, int c) { return false; }
};

loadAdminPayments(model, "All");

filter.addActionListener(e ->
        loadAdminPayments(model, filter.getSelectedItem().toString()));

JTable table = new JTable(model);
styleTable(table);
table.getColumnModel().getColumn(8).setCellRenderer(new StatusRenderer());
```

```
    JPanel actions = new JPanel(new FlowLayout(FlowLayout.RIGHT));
    JButton approve = button("Approve / Mark Paid", color1);
    JButton reject = button("Reject", RED);
    JButton archive = button("Archive", DARK);

    approve.addActionListener(e -> updatePaymentStatus(table, model, "Paid"));
    reject.addActionListener(e -> updatePaymentStatus(table, model, "Rejected"));
    archive.addActionListener(e -> deletePayment(table, model));

    actions.add(approve);
    actions.add(reject);
    actions.add(archive);

    p.add(new JScrollPane(table), BorderLayout.CENTER);
    p.add(actions, BorderLayout.SOUTH);
    return p;
}

/** Parameterized — no string concatenation of the status filter. */
private void loadAdminPayments(DefaultTableModel model, String filter) {
    model.setRowCount(0);

    StringBuilder sql = new StringBuilder(
            "SELECT p.payment_id, p.payment_date, u.username, r.room_number, "
            + "p.description, p.amount, p.method, p.reference_no, p.status "
            + "FROM payments p JOIN users u ON p.user_id=u.user_id "
            + "LEFT JOIN rooms r ON p.room_id=r.room_id ");

    boolean filtered = filter != null && !"All".equals(filter);
```

```
    if (filtered) sql.append("WHERE p.status=? ");
    sql.append("ORDER BY p.payment_id DESC");

    try (PreparedStatement ps = connection.prepareStatement(sql.toString())) {
        if (filtered) ps.setString(1, filter);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                model.addRow(new Object[]{
                        rs.getInt("payment_id"),
                        rs.getDate("payment_date"),
                        rs.getString("username"),
                        rs.getString("room_number"),
                        rs.getString("description"),
                        "\u20B1" + rs.getBigDecimal("amount"),
                        rs.getString("method"),
                        rs.getString("reference_no"),
                        rs.getString("status")
                });
            }
        }
    } catch (SQLException e) {
        showError(e);
    }
}

private void updatePaymentStatus(JTable table, DefaultTableModel model, String status) {
    int row = table.getSelectedRow();
    if (row < 0) {
        JOptionPane.showMessageDialog(this, "Select a payment first.");
```

```
            return;
        }

        int id = Integer.parseInt(model.getValueAt(row, 0).toString());

        String sql = "UPDATE payments SET status=? WHERE payment_id=?";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setString(1, status);
            ps.setInt(2, id);
            ps.executeUpdate();
            refreshAdmin("Payments");
        } catch (SQLException e) {
            showError(e);
        }
    }

    /** Moves a payment into the archive instead of destroying it. */
    private void deletePayment(JTable table, DefaultTableModel model) {
        int row = table.getSelectedRow();
        if (row < 0) {
            JOptionPane.showMessageDialog(this, "Select a payment first.");
            return;
        }

        if (!confirm("""
                Move this payment to the archive?
                You can still find it under Transaction Archive.""")) return;

        int id = Integer.parseInt(model.getValueAt(row, 0).toString());
```

```
    try {
        connection.setAutoCommit(false);
        archivePayment(id, currentUsername);

        try (PreparedStatement ps = connection.prepareStatement(
                "DELETE FROM payments WHERE payment_id=?")) {
            ps.setInt(1, id);
            ps.executeUpdate();
        }

        connection.commit();
        connection.setAutoCommit(true);
        refreshAdmin("Payments");
    } catch (SQLException e) {
        rollback();
        showError(e);
    }
}

private void ensureArchiveTable() {
    if (connection == null) {
        archiveReady = false;
        return;
    }

    String sql = "CREATE TABLE IF NOT EXISTS payment_archive ("
            + "archive_id INT AUTO_INCREMENT PRIMARY KEY,"
            + "payment_id INT,"
```

```
            + "user_id INT,"
            + "room_id INT,"
            + "username VARCHAR(50),"
            + "room_number VARCHAR(20),"
            + "payment_date DATE,"
            + "description VARCHAR(255),"
            + "amount DECIMAL(10,2),"
            + "method VARCHAR(50),"
            + "reference_no VARCHAR(60),"
            + "status VARCHAR(20),"
            + "archived_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,"
            + "archived_by VARCHAR(60)"
            + ") ENGINE=InnoDB DEFAULT CHARSET=utf8mb4";

    try (Statement st = connection.createStatement()) {
        st.executeUpdate(sql);
        archiveReady = true;
    } catch (SQLException e) {
        archiveReady = false;
    }
}

private void archivePayment(int paymentId, String archivedBy) throws SQLException {
    if (!archiveReady) return;

    String sql = "INSERT INTO payment_archive "
            + "(payment_id,user_id,room_id,username,room_number,payment_date,"
            + "description,amount,method,reference_no,status,archived_by) "
            + "SELECT p.payment_id, p.user_id, p.room_id, u.username, r.room_number, "
```

```
            + "p.payment_date, p.description, p.amount, p.method, p.reference_no, "
            + "p.status, ? "
            + "FROM payments p "
            + "LEFT JOIN users u ON p.user_id=u.user_id "
            + "LEFT JOIN rooms r ON p.room_id=r.room_id "
            + "WHERE p.payment_id=?";

        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setString(1, archivedBy);
            ps.setInt(2, paymentId);
            ps.executeUpdate();
        }
    }

    private void loadArchive(DefaultTableModel model, String search) {
        model.setRowCount(0);
        if (!archiveReady) return;

        StringBuilder sql = new StringBuilder(
                "SELECT archive_id, payment_date, username, room_number, description, "
                + "amount, method, reference_no, status, archived_at, archived_by "
                + "FROM payment_archive ");

        boolean filtered = search != null && !search.trim().isEmpty();
        if (filtered) {
            sql.append("WHERE username LIKE ? OR description LIKE ? OR reference_no LIKE ?
");
        }
        sql.append("ORDER BY archive_id DESC");
```

```
try (PreparedStatement ps = connection.prepareStatement(sql.toString())) {
    if (filtered) {
        String pattern = "%" + search.trim() + "%";
        ps.setString(1, pattern);
        ps.setString(2, pattern);
        ps.setString(3, pattern);
    }
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            model.addRow(new Object[]{
                    rs.getInt("archive_id"),
                    rs.getDate("payment_date"),
                    rs.getString("username"),
                    rs.getString("room_number"),
                    rs.getString("description"),
                    rs.getBigDecimal("amount") == null
                            ? ""
                            : "\u20B1" + rs.getBigDecimal("amount"),
                    rs.getString("method"),
                    rs.getString("reference_no"),
                    rs.getString("status"),
                    rs.getTimestamp("archived_at"),
                    rs.getString("archived_by")
            });
        }
    }
} catch (SQLException e) {
    showError(e);
```

```
        }
    }

    private void restoreArchivedPayment(JTable table, DefaultTableModel model) {
        int row = table.getSelectedRow();
        if (row < 0) {
            JOptionPane.showMessageDialog(this,
                    "Select an archived transaction first.");
            return;
        }

        int archiveId = Integer.parseInt(model.getValueAt(row, 0).toString());

        if (!confirm("Restore this payment back to the active payment list?")) return;

        try {
            connection.setAutoCommit(false);

            try (PreparedStatement ps = connection.prepareStatement(
                    "INSERT INTO payments "
                    + "(user_id, room_id, payment_date, description, amount, method, status,
reference_no) "
                    + "SELECT user_id, room_id, payment_date, description, amount, method, status,
reference_no "
                    + "FROM payment_archive WHERE archive_id=?")) {
                ps.setInt(1, archiveId);
                ps.executeUpdate();
            }

            try (PreparedStatement ps = connection.prepareStatement(
```

```
                "DELETE FROM payment_archive WHERE archive_id=?")) {
            ps.setInt(1, archiveId);
            ps.executeUpdate();
        }

        connection.commit();
        connection.setAutoCommit(true);
        refreshAdmin("Archive");
    } catch (SQLException e) {
        rollback();
        showError(e);
    }
}

private void deleteArchived(JTable table, DefaultTableModel model, boolean all) {
    if (all) {
        if (!confirm("""
                    Permanently delete ALL archived transactions?
                    This cannot be undone.""")) return;

        try (PreparedStatement ps = connection.prepareStatement(
                "DELETE FROM payment_archive")) {
            int removed = ps.executeUpdate();
            JOptionPane.showMessageDialog(this,
                    removed + " archived record(s) deleted.");
            refreshAdmin("Archive");
        } catch (SQLException e) {
            showError(e);
        }
```

```
        return;
    }

    int row = table.getSelectedRow();
    if (row < 0) {
        JOptionPane.showMessageDialog(this,
                "Select an archived transaction first.");
        return;
    }

    int archiveId = Integer.parseInt(model.getValueAt(row, 0).toString());

    if (!confirm("Permanently delete this archived record?")) return;

    try (PreparedStatement ps = connection.prepareStatement(
            "DELETE FROM payment_archive WHERE archive_id=?")) {
        ps.setInt(1, archiveId);
        ps.executeUpdate();
        refreshAdmin("Archive");
    } catch (SQLException e) {
        showError(e);
    }
}

private JPanel adminArchive() {
    JPanel p = basePanel("Transaction Archive",
            "History of removed payments. Restore them or delete permanently.");

    JPanel top = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 5));
```

```
top.setOpaque(false);
top.add(label("Search:", true));

JTextField search = new JTextField(22);
styleInput(search);
search.setPreferredSize(new Dimension(240, 32));
search.setMaximumSize(new Dimension(240, 32));
top.add(search);

JLabel hint = new JLabel("(tenant, description or reference no.)");
hint.setFont(new Font("Segoe UI", Font.PLAIN, 11));
hint.setForeground(GRAY);
top.add(hint);

setPageTop(p, top);

String[] columns = {"Archive ID", "Payment Date", "Tenant", "Room",
        "Description", "Amount", "Method", "Reference", "Status",
        "Archived At", "Archived By"};
DefaultTableModel model = new DefaultTableModel(columns, 0) {
    @Override
    public boolean isCellEditable(int r, int c) { return false; }
};

loadArchive(model, "");

search.getDocument().addDocumentListener(new DocumentListener() {
    @Override
    public void insertUpdate(DocumentEvent e) { loadArchive(model, search.getText()); }
```

```
        @Override
        public void removeUpdate(DocumentEvent e) { loadArchive(model, search.getText()); }

        @Override
        public void changedUpdate(DocumentEvent e) { loadArchive(model, search.getText()); }
    });

    JTable table = new JTable(model);
    styleTable(table);

    JPanel actions = new JPanel(new FlowLayout(FlowLayout.RIGHT));
    JButton restore = button("Restore Selected", BLUE);
    JButton deleteOne = button("Delete Selected", RED);
    JButton deleteAll = button("Delete All", DARK);

    restore.addActionListener(e -> restoreArchivedPayment(table, model));
    deleteOne.addActionListener(e -> deleteArchived(table, model, false));
    deleteAll.addActionListener(e -> deleteArchived(table, model, true));

    actions.add(restore);
    actions.add(deleteOne);
    actions.add(deleteAll);

    p.add(new JScrollPane(table), BorderLayout.CENTER);
    p.add(actions, BorderLayout.SOUTH);
    return p;
}
```

```
private JPanel adminUsers() {
    JPanel p = basePanel("Tenant & User Management",
            "Add, edit, deactivate, activate or remove accounts.");

    JPanel controls = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 5));
    controls.setOpaque(false);

    controls.add(label("Search:", true));
    JTextField searchField = new JTextField(20);
    styleInput(searchField);
    searchField.setPreferredSize(new Dimension(220, 32));
    searchField.setMaximumSize(new Dimension(220, 32));
    controls.add(searchField);

    controls.add(Box.createHorizontalStrut(15));
    controls.add(label("Role:", true));
    JComboBox<String> roleFilter = new JComboBox<>(
            new String[]{"All", "ADMIN", "TENANT"});
    controls.add(roleFilter);

    controls.add(Box.createHorizontalStrut(15));
    controls.add(label("Status:", true));
    JComboBox<String> statusFilter = new JComboBox<>(
            new String[]{"All", "Active", "Inactive"});
    controls.add(statusFilter);

    JPanel topWrapper = new JPanel(new BorderLayout());
    topWrapper.setOpaque(false);
```

```
topWrapper.add(controls, BorderLayout.WEST);

String[] cols = {"ID", "Username", "Full Name", "Email", "Phone",
        "Room", "Role", "Status"};
DefaultTableModel model = new DefaultTableModel(cols, 0) {
    @Override
    public boolean isCellEditable(int r, int c) { return false; }
};

JTable table = new JTable(model);
styleTable(table);

Runnable reload = () -> loadUsers(model,
        searchField.getText().trim(),
        roleFilter.getSelectedItem().toString(),
        statusFilter.getSelectedItem().toString());

reload.run();

searchField.getDocument().addDocumentListener(new DocumentListener() {
    @Override
    public void insertUpdate(DocumentEvent e) { reload.run(); }
    @Override
    public void removeUpdate(DocumentEvent e) { reload.run(); }
    @Override
    public void changedUpdate(DocumentEvent e) { reload.run(); }
});
roleFilter.addActionListener(e -> reload.run());
statusFilter.addActionListener(e -> reload.run());
```

```
    JPanel actions = new JPanel(new FlowLayout(FlowLayout.RIGHT));
    JButton add = button("Add User", color1);
    JButton edit = button("Edit Selected", BLUE);
    JButton activate = button("Activate", GREEN);
    JButton deactivate = button("Deactivate", ORANGE);
    JButton delete = button("Delete", RED);

    add.addActionListener(e -> addUser());
    edit.addActionListener(e -> editUser(table, model));
    activate.addActionListener(e -> setUserStatus(table, model, "Active"));
    deactivate.addActionListener(e -> setUserStatus(table, model, "Inactive"));
    delete.addActionListener(e -> deleteUser(table, model));

    actions.add(add);
    actions.add(edit);
    actions.add(activate);
    actions.add(deactivate);
    actions.add(delete);

    setPageTop(p, topWrapper);
    p.add(new JScrollPane(table), BorderLayout.CENTER);
    p.add(actions, BorderLayout.SOUTH);
    return p;
}

private void loadUsers(DefaultTableModel model, String search, String role, String status) {
    model.setRowCount(0);
    StringBuilder sql = new StringBuilder(
```

```
        "SELECT u.user_id, u.username, u.full_name, u.email, u.phone, "
        + "r.room_number, u.role, u.status "
        + "FROM users u LEFT JOIN rooms r ON u.room_id=r.room_id WHERE 1=1 ");

if (!search.isEmpty())
    sql.append("AND (u.username LIKE ? OR u.full_name LIKE ? OR u.email LIKE ?) ");
if (!"All".equals(role)) sql.append("AND u.role=? ");
if (!"All".equals(status)) sql.append("AND u.status=? ");
sql.append("ORDER BY u.user_id");

try (PreparedStatement ps = connection.prepareStatement(sql.toString())) {
    int parameter = 1;
    if (!search.isEmpty()) {
        String pattern = "%" + search + "%";
        ps.setString(parameter++, pattern);
        ps.setString(parameter++, pattern);
        ps.setString(parameter++, pattern);
    }
    if (!"All".equals(role)) ps.setString(parameter++, role);
    if (!"All".equals(status)) ps.setString(parameter, status);
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            model.addRow(new Object[]{
                    rs.getInt("user_id"),
                    rs.getString("username"),
                    rs.getString("full_name"),
                    rs.getString("email"),
                    rs.getString("phone"),
                    rs.getString("room_number"),
```

```
                    rs.getString("role"),
                    rs.getString("status")
                });
            }
        }
    } catch (SQLException e) {
        showError(e);
    }
}

private void addUser() {
    JTextField username = new JTextField();
    JPasswordField password = new JPasswordField();
    JTextField name = new JTextField();
    JTextField email = new JTextField();
    JTextField phone = new JTextField();
    JTextField address = new JTextField();

    JComboBox<String> role = new JComboBox<>(new String[]{"TENANT", "ADMIN"});
    JComboBox<RoomItem> room = roomCombo(true);
    JComboBox<String> status = new JComboBox<>(new String[]{"Active", "Inactive"});

    Object[] fields = {
        "Username:", username,
        "Password:", password,
        "Full Name:", name,
        "Email:", email,
        "Phone:", phone,
        "Address:", address,
```

```
        "Role:", role,
        "Assign Room:", room,
        "Status:", status
};

int result = JOptionPane.showConfirmDialog(this, fields,
        "Add User", JOptionPane.OK_CANCEL_OPTION);

if (result != JOptionPane.OK_OPTION) return;

String usernameValue = username.getText().trim();
String nameValue = name.getText().trim();
String emailValue = email.getText().trim();
String phoneValue = phone.getText().trim();
String addressValue = address.getText().trim();

char[] pw = password.getPassword();
try {
    if (!validText(usernameValue, 50, "Username") ||
            !validText(nameValue, 100, "Full name") ||
            !validPassword(pw) ||
            !validEmail(emailValue) ||
            !validOptionalText(phoneValue, 20, "Phone") ||
            !validOptionalText(addressValue, 255, "Address")) return;

    try {
        connection.setAutoCommit(false);

        int roomId = room.getSelectedItem() == null ? 0 :
```

```
            ((RoomItem) room.getSelectedItem()).id;

        String sql = "INSERT INTO users "
                + "(username,password,role,full_name,email,phone,address,room_id,status) "
                + "VALUES (?,?,?,?,?,?,?,?,?)";

        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setString(1, usernameValue);
            ps.setString(2, hashPassword(pw));
            ps.setString(3, role.getSelectedItem().toString());
            ps.setString(4, nameValue);
            ps.setString(5, emailValue);
            ps.setString(6, phoneValue);
            ps.setString(7, addressValue);
            if (roomId == 0) ps.setNull(8, Types.INTEGER);
            else ps.setInt(8, roomId);
            ps.setString(9, status.getSelectedItem().toString());
            ps.executeUpdate();
        }

        if (roomId != 0 && "TENANT".equals(role.getSelectedItem().toString())) {
            changeRoomOccupancy(roomId, 1);
        }

        connection.commit();
        connection.setAutoCommit(true);
        refreshAdmin("Users");
    } catch (SQLException e) {
        rollback();
```

```
            showError(e);
        }
    } finally {
        java.util.Arrays.fill(pw, '\0');
    }
}

private void editUser(JTable table, DefaultTableModel model) {
    int row = table.getSelectedRow();
    if (row < 0) {
        JOptionPane.showMessageDialog(this, "Select a user first.");
        return;
    }

    int userId = Integer.parseInt(model.getValueAt(row, 0).toString());

    try {
        String sql = "SELECT username,role,full_name,email,phone,address,room_id,status "
                + "FROM users WHERE user_id=?";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setInt(1, userId);
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) return;

                JTextField username = new JTextField(rs.getString("username"));
                JPasswordField password = new JPasswordField();
                password.setToolTipText("Leave blank to keep current password");
                JTextField name = new JTextField(rs.getString("full_name"));
                JTextField email = new JTextField(rs.getString("email"));
```

```
JTextField phone = new JTextField(rs.getString("phone"));
JTextField address = new JTextField(rs.getString("address"));

JComboBox<String> role = new JComboBox<>(
        new String[]{"TENANT", "ADMIN"});
role.setSelectedItem(rs.getString("role"));

JComboBox<RoomItem> room = roomCombo(true);
int oldRoom = rs.getInt("room_id");
if (rs.wasNull()) oldRoom = 0;

if (oldRoom != 0 && !containsRoom(room, oldRoom)) {
    room.addItem(new RoomItem(oldRoom, "Current Room (Full)"));
}
selectRoom(room, oldRoom);

JComboBox<String> status = new JComboBox<>(
        new String[]{"Active", "Inactive"});
status.setSelectedItem(rs.getString("status"));

Object[] fields = {
        "Username:", username,
        "New Password (blank = keep current):", password,
        "Full Name:", name,
        "Email:", email,
        "Phone:", phone,
        "Address:", address,
        "Role:", role,
        "Assign Room:", room,
```

```
        "Status:", status
};

int result = JOptionPane.showConfirmDialog(this, fields,
        "Edit User", JOptionPane.OK_CANCEL_OPTION);

if (result != JOptionPane.OK_OPTION) return;

String usernameValue = username.getText().trim();
String nameValue = name.getText().trim();
String emailValue = email.getText().trim();
String phoneValue = phone.getText().trim();
String addressValue = address.getText().trim();

char[] newPw = password.getPassword();
boolean changePw = newPw.length > 0;

if (!validText(usernameValue, 50, "Username") ||
        !validText(nameValue, 100, "Full name") ||
        !validEmail(emailValue) ||
        !validOptionalText(phoneValue, 20, "Phone") ||
        !validOptionalText(addressValue, 255, "Address")) {
    java.util.Arrays.fill(newPw, '\0');
    return;
}
if (changePw && !validPassword(newPw)) {
    java.util.Arrays.fill(newPw, '\0');
    return;
}
```

```
RoomItem selected = (RoomItem) room.getSelectedItem();
int newRoom = selected == null ? 0 : selected.id;

connection.setAutoCommit(false);

String oldRole = model.getValueAt(row, 6).toString();
String newRole = role.getSelectedItem().toString();

if ("TENANT".equals(newRole) && newRoom != 0
        && oldRoom != newRoom && !canAddToRoom(newRoom)) {
    connection.setAutoCommit(true);
    java.util.Arrays.fill(newPw, '\0');
    JOptionPane.showMessageDialog(this,
            "The selected room is full.");
    return;
}

String update;
if (changePw) {
    update = "UPDATE users SET username=?, password=?, role=?, "
            + "full_name=?, email=?, phone=?, address=?, room_id=?, status=? "
            + "WHERE user_id=?";
} else {
    update = "UPDATE users SET username=?, role=?, "
            + "full_name=?, email=?, phone=?, address=?, room_id=?, status=? "
            + "WHERE user_id=?";
}
```

```
try (PreparedStatement up = connection.prepareStatement(update)) {
    int i = 1;
    up.setString(i++, usernameValue);
    if (changePw) up.setString(i++, hashPassword(newPw));
    up.setString(i++, newRole);
    up.setString(i++, nameValue);
    up.setString(i++, emailValue);
    up.setString(i++, phoneValue);
    up.setString(i++, addressValue);

    if (newRoom == 0 || !"TENANT".equals(newRole))
        up.setNull(i++, Types.INTEGER);
    else
        up.setInt(i++, newRoom);

    up.setString(i++, status.getSelectedItem().toString());
    up.setInt(i, userId);
    up.executeUpdate();
} finally {
    java.util.Arrays.fill(newPw, '\0');
}

if ("TENANT".equals(oldRole) && oldRoom != 0
        && (!"TENANT".equals(newRole) || oldRoom != newRoom)) {
    changeRoomOccupancy(oldRoom, -1);
}

if ("TENANT".equals(newRole) && newRoom != 0
        && (!"TENANT".equals(oldRole) || oldRoom != newRoom)) {
```

```
                changeRoomOccupancy(newRoom, 1);
            }

            connection.commit();
            connection.setAutoCommit(true);
            refreshAdmin("Users");
        }
      }
    } catch (SQLException e) {
      rollback();
      showError(e);
    }
}

private void setUserStatus(JTable table, DefaultTableModel model, String status) {
    int row = table.getSelectedRow();
    if (row < 0) {
      JOptionPane.showMessageDialog(this, "Select a user first.");
      return;
    }

    int id = Integer.parseInt(model.getValueAt(row, 0).toString());

    if (id == currentUserId && "Inactive".equals(status)) {
      JOptionPane.showMessageDialog(this,
          "You cannot deactivate your own account.");
      return;
    }
```

```
    try (PreparedStatement ps =
                connection.prepareStatement(
                    "UPDATE users SET status=? WHERE user_id=?")) {
        ps.setString(1, status);
        ps.setInt(2, id);
        ps.executeUpdate();
        refreshAdmin("Users");
    } catch (SQLException e) {
        showError(e);
    }
}

private void deleteUser(JTable table, DefaultTableModel model) {
    int row = table.getSelectedRow();
    if (row < 0) {
        JOptionPane.showMessageDialog(this, "Select a user first.");
        return;
    }

    int id = Integer.parseInt(model.getValueAt(row, 0).toString());
    String username = model.getValueAt(row, 1).toString();

    if (id == currentUserId) {
        JOptionPane.showMessageDialog(this,
                "You cannot delete your own account.");
        return;
    }

    if (!confirm("Delete user '" + username
```

```
            + "'?\nRelated payments may also be deleted.")) return;

        try (PreparedStatement ps =
                connection.prepareStatement("DELETE FROM users WHERE user_id=?")) {
            ps.setInt(1, id);
            ps.executeUpdate();
            refreshAdmin("Users");
        } catch (SQLException e) {
            showError(this, e);
        }
    }


    private JPanel adminRooms() {
        JPanel p = basePanel("Room Management",
                "Add, edit or remove rooms. Occupancy is updated automatically.");

        String[] cols = {"ID", "Room", "Capacity", "Occupied",
                "Available", "Monthly Rent", "Status"};
        DefaultTableModel model = new DefaultTableModel(cols, 0) {
            @Override
            public boolean isCellEditable(int r, int c) { return false; }
        };

        String sql = "SELECT room_id, room_number, capacity, occupied, "
                + "(capacity-occupied) available, monthly_rent FROM rooms ORDER BY room_id";

        try (PreparedStatement ps = connection.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
```

```
    while (rs.next()) {
        int occ = rs.getInt("occupied");
        int cap = rs.getInt("capacity");
        model.addRow(new Object[]{
                rs.getInt("room_id"),
                rs.getString("room_number"),
                cap,
                occ,
                rs.getInt("available"),
                "\u20B1" + rs.getBigDecimal("monthly_rent"),
                occ >= cap ? "FULL" : "AVAILABLE"
        });
    }
} catch (SQLException e) {
    showError(e);
}

JTable table = new JTable(model);
styleTable(table);
table.getColumnModel().getColumn(6).setCellRenderer(new StatusRenderer());

JPanel actions = new JPanel(new FlowLayout(FlowLayout.RIGHT));
JButton add = button("Add Room", color1);
JButton edit = button("Edit Selected", BLUE);
JButton delete = button("Delete Room", RED);

add.addActionListener(e -> addRoom());
edit.addActionListener(e -> editRoom(table, model));
delete.addActionListener(e -> deleteRoom(table, model));
```

```
    actions.add(add);
    actions.add(edit);
    actions.add(delete);

    p.add(new JScrollPane(table), BorderLayout.CENTER);
    p.add(actions, BorderLayout.SOUTH);
    return p;
}

private void addRoom() {
    JTextField number = new JTextField();
    JTextField capacity = new JTextField("4");
    JTextField rent = new JTextField("3000");

    Object[] fields = {
            "Room Number:", number,
            "Capacity:", capacity,
            "Monthly Rent:", rent
    };

    int result = JOptionPane.showConfirmDialog(this, fields,
            "Add Room", JOptionPane.OK_CANCEL_OPTION);
    if (result != JOptionPane.OK_OPTION) return;

    try {
        int cap = Integer.parseInt(capacity.getText().trim());
        double monthlyRent = Double.parseDouble(rent.getText().trim());
```

```
        if (cap <= 0 || monthlyRent < 0 || number.getText().trim().isEmpty()
                || number.getText().trim().length() > 20) {
            JOptionPane.showMessageDialog(this, "Enter valid room information.");
            return;
        }

        String sql = "INSERT INTO rooms(room_number,capacity,occupied,monthly_rent) "
                + "VALUES(?,?,0,?)";

        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setString(1, number.getText().trim());
            ps.setInt(2, cap);
            ps.setDouble(3, monthlyRent);
            ps.executeUpdate();
        }

        refreshAdmin("Rooms");
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this,
                "Capacity and rent must be numbers.");
    } catch (SQLException e) {
        showError(e);
    }
}

private void editRoom(JTable table, DefaultTableModel model) {
    int row = table.getSelectedRow();
    if (row < 0) {
        JOptionPane.showMessageDialog(this, "Select a room first.");
```

```
    return;
}

int id = Integer.parseInt(model.getValueAt(row, 0).toString());
int occupied = Integer.parseInt(model.getValueAt(row, 3).toString());

JTextField number = new JTextField(model.getValueAt(row, 1).toString());
JTextField capacity = new JTextField(model.getValueAt(row, 2).toString());
JTextField rent = new JTextField(
        model.getValueAt(row, 5).toString().replace("\u20B1", ""));

Object[] fields = {
        "Room Number:", number,
        "Capacity (must be >= " + occupied + "):", capacity,
        "Monthly Rent:", rent
};

int result = JOptionPane.showConfirmDialog(this, fields,
        "Edit Room", JOptionPane.OK_CANCEL_OPTION);
if (result != JOptionPane.OK_OPTION) return;

try {
    int cap = Integer.parseInt(capacity.getText().trim());
    double monthlyRent = Double.parseDouble(rent.getText().trim());

    if (cap < occupied || cap <= 0 || monthlyRent < 0) {
        JOptionPane.showMessageDialog(this,
                "Capacity cannot be lower than current occupancy.");
        return;
```

```
        }

        String sql = "UPDATE rooms SET room_number=?, capacity=?, monthly_rent=? "
                + "WHERE room_id=?";

        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setString(1, number.getText().trim());
            ps.setInt(2, cap);
            ps.setDouble(3, monthlyRent);
            ps.setInt(4, id);
            ps.executeUpdate();
        }

        refreshAdmin("Rooms");
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this,
                "Capacity and rent must be numbers.");
    } catch (SQLException e) {
        showError(e);
    }
}

private void deleteRoom(JTable table, DefaultTableModel model) {
    int row = table.getSelectedRow();
    if (row < 0) {
        JOptionPane.showMessageDialog(this, "Select a room first.");
        return;
    }
```

```
    int id = Integer.parseInt(model.getValueAt(row, 0).toString());
    int occupied = Integer.parseInt(model.getValueAt(row, 3).toString());

    if (occupied > 0) {
        JOptionPane.showMessageDialog(this,
                "This room still has tenants. Reassign them before deleting.");
        return;
    }

    if (!confirm("Delete this room?")) return;

    try (PreparedStatement ps =
                 connection.prepareStatement("DELETE FROM rooms WHERE room_id=?")) {
        ps.setInt(1, id);
        ps.executeUpdate();
        refreshAdmin("Rooms");
    } catch (SQLException e) {
        showError(e);
    }
}


private JPanel adminApplications() {
    JPanel p = basePanel("Room Applications",
            "Accept or reject tenant room requests.");

    String[] columns = {"ID", "Tenant", "Target Room",
            "Move-in Date", "Status", "Message"};
    DefaultTableModel model = new DefaultTableModel(columns, 0) {
```

```
    @Override
    public boolean isCellEditable(int r, int c) { return false; }
};

String sql = "SELECT a.application_id, u.username, r.room_number, "
        + "a.move_in_date, a.status, a.message "
        + "FROM applications a JOIN users u ON a.user_id=u.user_id "
        + "JOIN rooms r ON a.room_id=r.room_id "
        + "ORDER BY a.application_id DESC";

try (PreparedStatement ps = connection.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        model.addRow(new Object[]{
                rs.getInt("application_id"),
                rs.getString("username"),
                rs.getString("room_number"),
                rs.getDate("move_in_date"),
                rs.getString("status"),
                rs.getString("message")
        });
    }
} catch (SQLException e) {
    showError(e);
}

JTable table = new JTable(model);
styleTable(table);
table.getColumnModel().getColumn(4).setCellRenderer(new StatusRenderer());
```

```
    JPanel buttons = new JPanel(new FlowLayout(FlowLayout.RIGHT));
    JButton accept = button("Accept", color1);
    JButton reject = button("Reject", RED);
    JButton delete = button("Delete", DARK);

    accept.addActionListener(e -> processApplication(table, model, true));
    reject.addActionListener(e -> processApplication(table, model, false));
    delete.addActionListener(e -> deleteApplication(table, model));

    buttons.add(accept);
    buttons.add(reject);
    buttons.add(delete);

    p.add(new JScrollPane(table), BorderLayout.CENTER);
    p.add(buttons, BorderLayout.SOUTH);
    return p;
}

private void processApplication(JTable table, DefaultTableModel model,
                                boolean accepted) {
    int row = table.getSelectedRow();
    if (row < 0) {
        JOptionPane.showMessageDialog(this, "Select an application first.");
        return;
    }

    int id = Integer.parseInt(model.getValueAt(row, 0).toString());
```

```
try {
    connection.setAutoCommit(false);

    int userId;
    int roomId;

    String find = "SELECT user_id, room_id, status "
            + "FROM applications WHERE application_id=?";
    try (PreparedStatement ps = connection.prepareStatement(find)) {
        ps.setInt(1, id);
        try (ResultSet rs = ps.executeQuery()) {
            if (!rs.next()) {
                connection.setAutoCommit(true);
                return;
            }
            userId = rs.getInt("user_id");
            roomId = rs.getInt("room_id");
            if (!"Pending".equalsIgnoreCase(rs.getString("status"))) {
                connection.setAutoCommit(true);
                JOptionPane.showMessageDialog(this,
                        "This application has already been processed.");
                return;
            }
        }
    }

    if (accepted) {
        int available = 0;
        try (PreparedStatement ps = connection.prepareStatement(
```

```
        "SELECT capacity-occupied FROM rooms "
        + "WHERE room_id=? FOR UPDATE")) {
    ps.setInt(1, roomId);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) available = rs.getInt(1);
    }
}

if (available <= 0) {
    connection.rollback();
    connection.setAutoCommit(true);
    JOptionPane.showMessageDialog(this,
            "This room is already full.");
    return;
}

int oldRoom = 0;
try (PreparedStatement ps = connection.prepareStatement(
        "SELECT room_id FROM users WHERE user_id=? FOR UPDATE")) {
    ps.setInt(1, userId);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
            oldRoom = rs.getInt(1);
            if (rs.wasNull()) oldRoom = 0;
        }
    }
}

if (oldRoom != 0 && oldRoom != roomId)
```

```
            changeRoomOccupancy(oldRoom, -1);

        try (PreparedStatement ps = connection.prepareStatement(
                "UPDATE users SET room_id=?, status='Active' WHERE user_id=?")) {
            ps.setInt(1, roomId);
            ps.setInt(2, userId);
            ps.executeUpdate();
        }

        changeRoomOccupancy(roomId, 1);
    }

    try (PreparedStatement ps = connection.prepareStatement(
            "UPDATE applications SET status=? WHERE application_id=?")) {
        ps.setString(1, accepted ? "Approved" : "Rejected");
        ps.setInt(2, id);
        ps.executeUpdate();
    }

    connection.commit();
    connection.setAutoCommit(true);

    JOptionPane.showMessageDialog(this,
            accepted ? "Application approved." : "Application rejected.");
    refreshAdmin("Applications");

} catch (SQLException e) {
    rollback();
    showError(e);
```

```
    }
}

private void deleteApplication(JTable table, DefaultTableModel model) {
    int row = table.getSelectedRow();
    if (row < 0) return;
    if (!confirm("Delete this application record?")) return;

    int id = Integer.parseInt(model.getValueAt(row, 0).toString());

    try (PreparedStatement ps = connection.prepareStatement(
            "DELETE FROM applications WHERE application_id=?")) {
        ps.setInt(1, id);
        ps.executeUpdate();
        refreshAdmin("Applications");
    } catch (SQLException e) {
        showError(e);
    }
}



private JPanel rulesPanel(boolean admin) {
    JPanel p = basePanel(
            admin ? "Boarding House Rules & Policy" : "Rules & Guidelines",
            "House rules for all Ocampo's Residence tenants.");

    JPanel list = new JPanel();
    list.setLayout(new BoxLayout(list, BoxLayout.Y_AXIS));
    list.setBackground(WHITE);
```

```
list.setBorder(new EmptyBorder(10, 10, 10, 10));

String sql = "SELECT rule_id, title, description FROM rules ORDER BY rule_id";

try (PreparedStatement ps = connection.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        int id = rs.getInt("rule_id");

        JLabel title = new JLabel(rs.getString("title"));
        title.setFont(new Font("Segoe UI", Font.BOLD, 13));
        title.setForeground(DARK);

        JTextArea body = new JTextArea(rs.getString("description"));
        body.setFont(new Font("Segoe UI", Font.PLAIN, 11));
        body.setForeground(GRAY);
        body.setLineWrap(true);
        body.setWrapStyleWord(true);
        body.setEditable(false);
        body.setBackground(WHITE);

        JPanel card = new JPanel(new BorderLayout(0, 5));
        card.setBackground(WHITE);
        card.setBorder(BorderFactory.createCompoundBorder(
                BorderFactory.createLineBorder(new Color(0xE5, 0xE7, 0xEB)),
                new EmptyBorder(12, 14, 12, 14)));
        card.add(title, BorderLayout.NORTH);
        card.add(body, BorderLayout.CENTER);
```

```
            if (admin) {
                JButton edit = button("Edit", BLUE);
                edit.addActionListener(e -> editRule(id));
                card.add(edit, BorderLayout.EAST);
            }

            list.add(card);
            list.add(Box.createVerticalStrut(10));
        }
    } catch (SQLException e) {
        showError(e);
    }

    JPanel south = new JPanel(new FlowLayout(FlowLayout.RIGHT));
    if (admin) {
        JButton add = button("Add Rule", color1);
        add.addActionListener(e -> addRule());
        south.add(add);
    }

    p.add(new JScrollPane(list), BorderLayout.CENTER);
    if (admin) p.add(south, BorderLayout.SOUTH);
    return p;
}

private void addRule() {
    JTextField title = new JTextField();
    JTextArea description = new JTextArea(6, 30);
    description.setLineWrap(true);
```

```
    description.setWrapStyleWord(true);

    Object[] fields = {"Title:", title,
            "Description:", new JScrollPane(description)};
    int result = JOptionPane.showConfirmDialog(this, fields,
            "Add Rule", JOptionPane.OK_CANCEL_OPTION);

    if (result != JOptionPane.OK_OPTION) return;
    if (!validText(title.getText(), 100, "Rule title") ||
            !validText(description.getText(), 4000, "Rule description")) return;

    try (PreparedStatement ps = connection.prepareStatement(
            "INSERT INTO rules(title,description) VALUES(?,?)")) {
        ps.setString(1, title.getText().trim());
        ps.setString(2, description.getText().trim());
        ps.executeUpdate();
        refreshAdmin("Rules");
    } catch (SQLException e) {
        showError(e);
    }
}

private void editRule(int id) {
    try (PreparedStatement ps = connection.prepareStatement(
            "SELECT title,description FROM rules WHERE rule_id=?")) {
        ps.setInt(1, id);
        try (ResultSet rs = ps.executeQuery()) {
            if (!rs.next()) return;
```

```
        JTextField title = new JTextField(rs.getString("title"));
        JTextArea description = new JTextArea(
                rs.getString("description"), 6, 30);
        description.setLineWrap(true);
        description.setWrapStyleWord(true);

        Object[] fields = {"Title:", title, "Description:",
                new JScrollPane(description)};

        int result = JOptionPane.showConfirmDialog(this, fields,
                "Edit Rule", JOptionPane.OK_CANCEL_OPTION);

        if (result != JOptionPane.OK_OPTION) return;

        if (!validText(title.getText(), 100, "Rule title") ||
                !validText(description.getText(), 4000, "Rule description")) return;

        try (PreparedStatement up = connection.prepareStatement(
                "UPDATE rules SET title=?, description=? WHERE rule_id=?")) {
            up.setString(1, title.getText().trim());
            up.setString(2, description.getText().trim());
            up.setInt(3, id);
            up.executeUpdate();
        }

        refreshAdmin("Rules");
    }
} catch (SQLException e) {
    showError(e);
```

```
    }
}

private JPanel adminRecovery() {
    JPanel p = basePanel("Account Recovery & Status",
            "Reset tenant passwords and manage account status.");

    String[] cols = {"ID", "Tenant", "Username", "Email", "Status"};
    DefaultTableModel model = new DefaultTableModel(cols, 0) {
        @Override
        public boolean isCellEditable(int r, int c) { return false; }
    };

    String sql = "SELECT user_id, full_name, username, email, status "
            + "FROM users WHERE role='TENANT' ORDER BY user_id";

    try (PreparedStatement ps = connection.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            model.addRow(new Object[]{
                    rs.getInt("user_id"),
                    rs.getString("full_name"),
                    rs.getString("username"),
                    rs.getString("email"),
                    rs.getString("status")
            });
        }
    } catch (SQLException e) {
        showError(e);
```

```
    }

    JTable table = new JTable(model);
    styleTable(table);

    JButton reset = button("Reset Password", color1);
    JButton activate = button("Activate", GREEN);
    JButton deactivate = button("Deactivate", ORANGE);

    reset.addActionListener(e -> resetPassword(table, model));
    activate.addActionListener(e -> setUserStatus(table, model, "Active"));
    deactivate.addActionListener(e -> setUserStatus(table, model, "Inactive"));

    JPanel actions = new JPanel(new FlowLayout(FlowLayout.RIGHT));
    actions.add(reset);
    actions.add(activate);
    actions.add(deactivate);

    p.add(new JScrollPane(table), BorderLayout.CENTER);
    p.add(actions, BorderLayout.SOUTH);
    return p;
}

private void resetPassword(JTable table, DefaultTableModel model) {
    int row = table.getSelectedRow();
    if (row < 0) return;

    int id = Integer.parseInt(model.getValueAt(row, 0).toString());
    JPasswordField password = new JPasswordField();
```

```
    int result = JOptionPane.showConfirmDialog(this, password,
            "New Password", JOptionPane.OK_CANCEL_OPTION);

    if (result != JOptionPane.OK_OPTION) return;

    char[] pw = password.getPassword();
    try {
        if (!validPassword(pw)) return;

        try (PreparedStatement ps = connection.prepareStatement(
                "UPDATE users SET password=? WHERE user_id=?")) {
            ps.setString(1, hashPassword(pw));
            ps.setInt(2, id);
            ps.executeUpdate();
            JOptionPane.showMessageDialog(this, "Password reset successfully.");
        }
    } catch (SQLException e) {
        showError(e);
    } finally {
        java.util.Arrays.fill(pw, '\0');
    }
}

private void showTenant() {
    getContentPane().removeAll();
    setLayout(new BorderLayout());

    JPanel sidebar = new JPanel();
```

```
sidebar.setBackground(Themaindontforget);
sidebar.setPreferredSize(new Dimension(235, 780));
sidebar.setLayout(new BoxLayout(sidebar, BoxLayout.Y_AXIS));
sidebar.setBorder(new EmptyBorder(20, 15, 20, 15));

JLabel brand = new JLabel("Ocampo's Residence");
brand.setFont(new Font("Segoe UI", Font.BOLD, 16));
brand.setForeground(WHITE);
brand.setBorder(new EmptyBorder(0, 8, 30, 0));
sidebar.add(brand);

content = new JPanel(new CardLayout());
content.setBackground(BG);
cardLayout = (CardLayout) content.getLayout();

content.add(tenantDashboard(), "Dashboard");
content.add(tenantProfile(), "Profile");
content.add(tenantRules(), "Rules");
content.add(tenantPayments(), "Payments");
content.add(tenantApplication(), "Application");

addTenantNav(sidebar, "Dashboard", "Dashboard");
addTenantNav(sidebar, "My Profile", "Profile");
addTenantNav(sidebar, "House Rules", "Rules");
addTenantNav(sidebar, "My Payments", "Payments");
addTenantNav(sidebar, "Room Application", "Application");

sidebar.add(Box.createVerticalGlue());
```

```
    JButton logout = new JButton("Logout");
    logout.setForeground(RED);
    logout.setBackground(Themaindontforget);
    logout.setContentAreaFilled(false);
    logout.setBorderPainted(false);
    logout.setFocusPainted(false);
    logout.setAlignmentX(Component.LEFT_ALIGNMENT);
    logout.setCursor(new Cursor(Cursor.HAND_CURSOR));
    logout.addActionListener(e -> logout());
    sidebar.add(logout);

    add(sidebar, BorderLayout.WEST);
    add(content, BorderLayout.CENTER);
    cardLayout.show(content, "Dashboard");

    if (content != null) applyZoomToTree(content);
    revalidate();
    repaint();
}

private void addTenantNav(JPanel sidebar, String text, String page) {
    JButton b = new JButton(text);
    b.setMaximumSize(new Dimension(205, 40));
    b.setForeground(WHITE);
    b.setBackground(Themaindontforget);
    b.setFocusPainted(false);
    b.setBorderPainted(false);
    b.setOpaque(true);
    b.setHorizontalAlignment(SwingConstants.LEFT);
```

```
    b.setFont(new Font("Segoe UI", Font.PLAIN, 13));
    b.setBorder(new EmptyBorder(5, 10, 5, 10));

    b.addActionListener(e -> {
        cardLayout.show(content, page);
        selectNavButton(sidebar, b);
    });

    installNavHover(b);

    sidebar.add(b);
    sidebar.add(Box.createVerticalStrut(6));
}

private JPanel tenantDashboard() {
    JPanel p = basePanel("My Dashboard",
            "Welcome back, " + currentUsername + "!");

    JScrollPane scroll = new JScrollPane(buildTenantDashboardContent());
    scroll.setBorder(null);
    scroll.getVerticalScrollBar().setUnitIncrement(16);
    scroll.setOpaque(false);
    scroll.getViewport().setOpaque(false);

    p.add(scroll, BorderLayout.CENTER);
    return p;
}

private JPanel buildTenantDashboardContent() {
```

```
JPanel main = new JPanel();
main.setLayout(new BoxLayout(main, BoxLayout.Y_AXIS));
main.setOpaque(false);

JPanel metrics = new JPanel(new GridLayout(1, 6, 12, 0));
metrics.setOpaque(false);
metrics.setPreferredSize(new Dimension(0, 105));
metrics.setMaximumSize(new Dimension(Integer.MAX_VALUE, 105));

metrics.add(metric("MY ROOM", tenantRoom(),
        "Assigned room", "\uD83C\uDFE0", color1));
metrics.add(metric("MONTHLY RENT", tenantMonthlyRent(),
        "Per month", "\uD83D\uDCC5", BLUE));
metrics.add(metric("BALANCE DUE", tenantBalance(),
        "Outstanding", "\u20B1", ORANGE));
metrics.add(metric("TOTAL PAID", tenantTotalPaid(),
        "All time", "\u2713", GREEN));
metrics.add(metric("NEXT DUE", nextDue(),
        "Payment date", "\u23F0", PURPLE));
metrics.add(metric("STATUS", tenantStatus(), "Account", "\u25CF",
        "Active".equals(tenantStatus()) ? GREEN : RED));

main.add(metrics);
main.add(Box.createVerticalStrut(15));

JPanel progressCard = whiteCard();
progressCard.setLayout(new BorderLayout(0, 10));
progressCard.setMaximumSize(new Dimension(Integer.MAX_VALUE, 130));
progressCard.add(sectionTitle("Payment Progress"), BorderLayout.NORTH);
```

```
JPanel progressBody = new JPanel(new BorderLayout(0, 8));
progressBody.setOpaque(false);

double total = tenantTotalBilled();
double paid = tenantTotalPaidValue();
int pct = total > 0 ? (int) Math.round(paid / total * 100) : 0;

JProgressBar bar = new JProgressBar(0, 100);
bar.setValue(pct);
bar.setStringPainted(true);
bar.setString(pct + "% Paid");
bar.setFont(new Font("Segoe UI", Font.BOLD, 11));
bar.setForeground(GREEN);
bar.setBackground(new Color(0xE5, 0xE7, 0xEB));
bar.setPreferredSize(new Dimension(0, 28));

JLabel detail = new JLabel(String.format(
        "\u20B1%.2f paid of \u20B1%.2f total billed  |  Outstanding: \u20B1%.2f",
        paid, total, Math.max(0, total - paid)));
detail.setFont(new Font("Segoe UI", Font.PLAIN, 11));
detail.setForeground(GRAY);

progressBody.add(bar, BorderLayout.NORTH);
progressBody.add(detail, BorderLayout.SOUTH);
progressCard.add(progressBody, BorderLayout.CENTER);
main.add(progressCard);
main.add(Box.createVerticalStrut(15));
```

```
    JPanel row = new JPanel(new GridLayout(1, 2, 15, 0));
    row.setOpaque(false);
    row.setMaximumSize(new Dimension(Integer.MAX_VALUE, 200));

    row.add(roomDetailsCard());
    row.add(accountSummaryCard());

    main.add(row);
    main.add(Box.createVerticalStrut(15));

    JPanel row2 = new JPanel(new GridLayout(1, 2, 15, 0));
    row2.setOpaque(false);
    row2.setMaximumSize(new Dimension(Integer.MAX_VALUE, 240));

    row2.add(tenantRecentPaymentsCard());
    row2.add(tenantRecentApplicationsCard());

    main.add(row2);

    return main;
}

private JPanel roomDetailsCard() {
    JPanel panel = whiteCard();
    panel.setLayout(new BorderLayout(0, 8));
    panel.add(sectionTitle("My Room Details"), BorderLayout.NORTH);

    JPanel grid = new JPanel(new GridLayout(0, 2, 10, 6));
    grid.setOpaque(false);
```

```
    String sql = "SELECT r.room_number, r.capacity, r.occupied, r.monthly_rent "
            + "FROM users u LEFT JOIN rooms r ON u.room_id=r.room_id "
            + "WHERE u.user_id=?";

    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, currentUserId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next() && rs.getString("room_number") != null) {
                addInfoRow(grid, "Room Number", rs.getString("room_number"));
                addInfoRow(grid, "Capacity", String.valueOf(rs.getInt("capacity")));
                addInfoRow(grid, "Occupied", String.valueOf(rs.getInt("occupied")));
                addInfoRow(grid, "Monthly Rent",
                        "\u20B1" + rs.getBigDecimal("monthly_rent"));
            } else {
                JLabel none = new JLabel("No room assigned yet.");
                none.setFont(new Font("Segoe UI", Font.ITALIC, 12));
                none.setForeground(GRAY);
                panel.add(none, BorderLayout.CENTER);
                return panel;
            }
        }
    } catch (SQLException e) {
        showError(e);
    }

    panel.add(grid, BorderLayout.CENTER);
    return panel;
}
```

```
private JPanel accountSummaryCard() {
    JPanel panel = whiteCard();
    panel.setLayout(new BorderLayout(0, 8));
    panel.add(sectionTitle("Account Summary"), BorderLayout.NORTH);

    JPanel grid = new JPanel(new GridLayout(0, 2, 10, 6));
    grid.setOpaque(false);

    String sql = "SELECT username, full_name, email, phone, address, status, "
            + "DATE(created_at) join_date FROM users WHERE user_id=?";

    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, currentUserId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                addInfoRow(grid, "Username", rs.getString("username"));
                addInfoRow(grid, "Full Name", rs.getString("full_name"));
                addInfoRow(grid, "Email", rs.getString("email"));
                addInfoRow(grid, "Phone", rs.getString("phone"));
                addInfoRow(grid, "Status", rs.getString("status"));
                addInfoRow(grid, "Member Since", rs.getString("join_date"));
            }
        }
    } catch (SQLException e) {
        showError(e);
    }

    panel.add(grid, BorderLayout.CENTER);
```

```
    return panel;
}

private void addInfoRow(JPanel grid, String key, String value) {
    JLabel k = new JLabel(key + ":");
    k.setFont(new Font("Segoe UI", Font.BOLD, 11));
    k.setForeground(GRAY);

    JLabel v = new JLabel(value == null || value.isEmpty() ? "\u2014" : value);
    v.setFont(new Font("Segoe UI", Font.PLAIN, 12));
    v.setForeground(DARK);

    grid.add(k);
    grid.add(v);
}

private JPanel tenantRecentPaymentsCard() {
    JPanel panel = whiteCard();
    panel.setLayout(new BorderLayout(0, 8));
    panel.add(sectionTitle("My Recent Payments"), BorderLayout.NORTH);

    String[] cols = {"Date", "Description", "Amount", "Status"};
    DefaultTableModel model = new DefaultTableModel(cols, 0) {
        @Override
        public boolean isCellEditable(int r, int c) { return false; }
    };

    String sql = "SELECT payment_date, description, amount, status FROM payments "
            + "WHERE user_id=? ORDER BY payment_id DESC LIMIT 8";
```

```
    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, currentUserId);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                model.addRow(new Object[]{
                        rs.getDate("payment_date"),
                        rs.getString("description"),
                        "\u20B1" + rs.getBigDecimal("amount"),
                        rs.getString("status")
                });
            }
        }
    } catch (SQLException e) {
        showError(e);
    }

    JTable table = new JTable(model);
    styleTable(table);
    table.getColumnModel().getColumn(3).setCellRenderer(new StatusRenderer());
    panel.add(new JScrollPane(table), BorderLayout.CENTER);
    return panel;
}

private JPanel tenantRecentApplicationsCard() {
    JPanel panel = whiteCard();
    panel.setLayout(new BorderLayout(0, 8));
    panel.add(sectionTitle("My Recent Applications"), BorderLayout.NORTH);
```

```
String[] cols = {"Room", "Move-in", "Status"};
DefaultTableModel model = new DefaultTableModel(cols, 0) {
    @Override
    public boolean isCellEditable(int r, int c) { return false; }
};

String sql = "SELECT r.room_number, a.move_in_date, a.status "
        + "FROM applications a JOIN rooms r ON a.room_id=r.room_id "
        + "WHERE a.user_id=? ORDER BY a.application_id DESC LIMIT 8";

try (PreparedStatement ps = connection.prepareStatement(sql)) {
    ps.setInt(1, currentUserId);
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            model.addRow(new Object[]{
                    rs.getString("room_number"),
                    rs.getDate("move_in_date"),
                    rs.getString("status")
            });
        }
    }
} catch (SQLException e) {
    showError(e);
}

JTable table = new JTable(model);
styleTable(table);
table.getColumnModel().getColumn(2).setCellRenderer(new StatusRenderer());
panel.add(new JScrollPane(table), BorderLayout.CENTER);
```

```
    return panel;
}

private JPanel tenantProfile() {
    JPanel p = basePanel("My Profile",
            "View your account and update personal information.");

    JScrollPane scroll = new JScrollPane(buildTenantProfileContent());
    scroll.setBorder(null);
    scroll.getVerticalScrollBar().setUnitIncrement(16);
    scroll.setOpaque(false);
    scroll.getViewport().setOpaque(false);

    p.add(scroll, BorderLayout.CENTER);
    return p;
}

private JPanel buildTenantProfileContent() {
    JPanel wrapper = new JPanel();
    wrapper.setLayout(new BoxLayout(wrapper, BoxLayout.Y_AXIS));
    wrapper.setOpaque(false);

    JPanel accountCard = whiteCard();
    accountCard.setLayout(new BorderLayout(0, 8));
    accountCard.setMaximumSize(new Dimension(Integer.MAX_VALUE, 200));
    accountCard.add(sectionTitle("Account Information (Read-only)"),
            BorderLayout.NORTH);

    JPanel accountGrid = new JPanel(new GridLayout(0, 2, 10, 6));
```

```
accountGrid.setOpaque(false);

try (PreparedStatement ps = connection.prepareStatement(
        "SELECT username, role, status, DATE(created_at) join_date "
        + "FROM users WHERE user_id=?")) {
    ps.setInt(1, currentUserId);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
            addInfoRow(accountGrid, "User ID", String.valueOf(currentUserId));
            addInfoRow(accountGrid, "Username", rs.getString("username"));
            addInfoRow(accountGrid, "Role", rs.getString("role"));
            addInfoRow(accountGrid, "Account Status", rs.getString("status"));
            addInfoRow(accountGrid, "Member Since", rs.getString("join_date"));
        }
    }
} catch (SQLException e) {
    showError(e);
}

accountCard.add(accountGrid, BorderLayout.CENTER);
wrapper.add(accountCard);
wrapper.add(Box.createVerticalStrut(15));

JPanel formCard = whiteCard();
formCard.setLayout(new BorderLayout(0, 10));
formCard.setMaximumSize(new Dimension(Integer.MAX_VALUE, 350));
formCard.add(sectionTitle("Edit Personal Information"), BorderLayout.NORTH);

JPanel form = new JPanel(new GridBagLayout());
```

```
form.setOpaque(false);

JTextField name = new JTextField();
JTextField email = new JTextField();
JTextField phone = new JTextField();
JTextField address = new JTextField();

try (PreparedStatement ps = connection.prepareStatement(
        "SELECT full_name,email,phone,address FROM users WHERE user_id=?")) {
    ps.setInt(1, currentUserId);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
            name.setText(rs.getString("full_name"));
            email.setText(rs.getString("email"));
            phone.setText(rs.getString("phone"));
            address.setText(rs.getString("address"));
        }
    }
} catch (SQLException e) {
    showError(e);
}

GridBagConstraints g = new GridBagConstraints();
g.insets = new Insets(7, 7, 7, 7);
g.fill = GridBagConstraints.HORIZONTAL;
g.weightx = 1;

addFormRow(form, g, 0, "Full Name", name);
addFormRow(form, g, 1, "Email", email);
```

```
addFormRow(form, g, 2, "Phone", phone);
addFormRow(form, g, 3, "Address", address);

JButton save = button("Save Changes", color1);
save.addActionListener(e -> {
    if (!validText(name.getText(), 100, "Full name") ||
            !validEmail(email.getText()) ||
            !validOptionalText(phone.getText(), 20, "Phone") ||
            !validOptionalText(address.getText(), 255, "Address")) return;

    try {
        String sql = "UPDATE users SET full_name=?, email=?, phone=?, address=? "
                + "WHERE user_id=?";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setString(1, name.getText().trim());
            ps.setString(2, email.getText().trim());
            ps.setString(3, phone.getText().trim());
            ps.setString(4, address.getText().trim());
            ps.setInt(5, currentUserId);
            ps.executeUpdate();
        }
        JOptionPane.showMessageDialog(this, "Profile updated.");
        showTenant();
        cardLayout.show(content, "Profile");
    } catch (SQLException ex) {
        showError(ex);
    }
});
```

```
g.gridy = 4;
g.gridx = 1;
g.weightx = 0;
form.add(save, g);

formCard.add(form, BorderLayout.CENTER);
wrapper.add(formCard);
wrapper.add(Box.createVerticalStrut(15));

JPanel pwCard = whiteCard();
pwCard.setLayout(new BorderLayout(0, 10));
pwCard.setMaximumSize(new Dimension(Integer.MAX_VALUE, 220));
pwCard.add(sectionTitle("Change Password"), BorderLayout.NORTH);

JPanel pwForm = new JPanel(new GridBagLayout());
pwForm.setOpaque(false);

JPasswordField oldPw = new JPasswordField();
JPasswordField newPw = new JPasswordField();
JPasswordField confirmPw = new JPasswordField();

GridBagConstraints g2 = new GridBagConstraints();
g2.insets = new Insets(7, 7, 7, 7);
g2.fill = GridBagConstraints.HORIZONTAL;
g2.weightx = 1;

addFormRow(pwForm, g2, 0, "Current Password", oldPw);
addFormRow(pwForm, g2, 1, "New Password", newPw);
addFormRow(pwForm, g2, 2, "Confirm New Password", confirmPw);
```

```
JButton changePw = button("Change Password", BLUE);
changePw.addActionListener(e -> {
    char[] oldChars = oldPw.getPassword();
    char[] newChars = newPw.getPassword();
    char[] confChars = confirmPw.getPassword();

    try {
        if (!validPassword(newChars)) return;
        if (!MessageDigest.isEqual(
                new String(newChars).getBytes(StandardCharsets.UTF_8),
                new String(confChars).getBytes(StandardCharsets.UTF_8))) {
            JOptionPane.showMessageDialog(this,
                    "New passwords do not match.");
            return;
        }

        try (PreparedStatement ps = connection.prepareStatement(
                "SELECT password FROM users WHERE user_id=?")) {
            ps.setInt(1, currentUserId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next() && !verifyPassword(oldChars, rs.getString("password"))) {
                    JOptionPane.showMessageDialog(this,
                            "Current password is incorrect.");
                    return;
                }
            }
        }
```

```
        try (PreparedStatement up = connection.prepareStatement(
                "UPDATE users SET password=? WHERE user_id=?")) {
            up.setString(1, hashPassword(newChars));
            up.setInt(2, currentUserId);
            up.executeUpdate();
        }
        JOptionPane.showMessageDialog(this,
                "Password changed successfully.");
        oldPw.setText("");
        newPw.setText("");
        confirmPw.setText("");
    } catch (SQLException ex) {
        showError(ex);
    } finally {
        java.util.Arrays.fill(oldChars, '\0');
        java.util.Arrays.fill(newChars, '\0');
        java.util.Arrays.fill(confChars, '\0');
    }
});

g2.gridy = 3;
g2.gridx = 1;
g2.weightx = 0;
pwForm.add(changePw, g2);

pwCard.add(pwForm, BorderLayout.CENTER);
wrapper.add(pwCard);

return wrapper;
```

```
}

private void addFormRow(JPanel form, GridBagConstraints g, int row,
                        String name, JComponent field) {
    g.gridy = row;
    g.gridx = 0;
    g.weightx = 0;
    form.add(label(name, true), g);

    g.gridx = 1;
    g.weightx = 1;
    field.setPreferredSize(new Dimension(400, 34));
    form.add(field, g);
}

private JPanel tenantRules() {
    return rulesPanel(false);
}

private JPanel tenantPayments() {
    JPanel p = basePanel("My Payments & Invoices",
            "View payment history and submit a payment.");

    JPanel summary = new JPanel(new GridLayout(1, 4, 12, 0));
    summary.setOpaque(false);
    summary.setPreferredSize(new Dimension(0, 95));

    summary.add(metric("TOTAL BILLED",
            "\u20B1" + String.format("%.2f", tenantTotalBilled()),
```

```
        "All invoices", "\uD83D\uDCC4", color1));
summary.add(metric("TOTAL PAID", tenantTotalPaid(),
        "Completed", "\u2713", GREEN));
summary.add(metric("BALANCE", tenantBalance(),
        "Outstanding", "\u20B1", ORANGE));
summary.add(metric("PENDING", countMyPayments("Pending"),
        "Awaiting", "\u23F3", BLUE));

setPageTop(p, summary);

String[] columns = {"ID", "Date", "Description", "Amount",
        "Method", "Status", "Reference"};
DefaultTableModel model = new DefaultTableModel(columns, 0) {
    @Override
    public boolean isCellEditable(int r, int c) { return false; }
};

String sql = "SELECT payment_id,payment_date,description,amount,method,"
        + "status,reference_no FROM payments "
        + "WHERE user_id=? ORDER BY payment_id DESC";

try (PreparedStatement ps = connection.prepareStatement(sql)) {
    ps.setInt(1, currentUserId);
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            model.addRow(new Object[]{
                    rs.getInt("payment_id"),
                    rs.getDate("payment_date"),
                    rs.getString("description"),
```

```
                    "\u20B1" + rs.getBigDecimal("amount"),
                    rs.getString("method"),
                    rs.getString("status"),
                    rs.getString("reference_no")
                });
            }
        }
    } catch (SQLException e) {
        showError(e);
    }

    JTable table = new JTable(model);
    styleTable(table);
    table.getColumnModel().getColumn(5).setCellRenderer(new StatusRenderer());

    JPanel actions = new JPanel(new FlowLayout(FlowLayout.RIGHT));
    JButton pay = button("Submit Payment", color1);
    JButton cancel = button("Cancel Pending Payment", RED);

    pay.addActionListener(e -> submitPayment());
    cancel.addActionListener(e -> cancelPayment(table, model));

    actions.add(pay);
    actions.add(cancel);

    p.add(new JScrollPane(table), BorderLayout.CENTER);
    p.add(actions, BorderLayout.SOUTH);
    return p;
}
```

```
private String countMyPayments(String status) {
    String sql = "SELECT COUNT(*) FROM payments WHERE user_id=? AND status=?";
    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, currentUserId);
        ps.setString(2, status);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return String.valueOf(rs.getInt(1));
        }
    } catch (SQLException e) {
        showError(e);
    }
    return "0";
}

private void submitPayment() {
    JRadioButton fullOption = new JRadioButton(String.format(
            "Full Payment \u2014 \u20B1%,.2f", FULL_PAYMENT_AMOUNT));
    JRadioButton partialOption = new JRadioButton(String.format(
            "Partial Payment \u2014 \u20B1%,.2f", PARTIAL_PAYMENT_AMOUNT));
    JRadioButton minimalOption = new JRadioButton(String.format(
            "Minimal Payment \u2014 \u20B1%,.2f", MINIMAL_PAYMENT_AMOUNT));
    JRadioButton advanceOption = new JRadioButton(String.format(
            "Advance 3 Months \u2014 \u20B1%,.2f  (\u20B1%,.0f off)",
            ADVANCE_3MO_AMOUNT, ADVANCE_3MO_DISCOUNT));

    JRadioButton[] options = {fullOption, partialOption, minimalOption, advanceOption};
    for (JRadioButton rb : options) {
        rb.setFont(new Font("Segoe UI", Font.PLAIN, 12));
```

```
    rb.setForeground(DARK);
    rb.setOpaque(false);
    rb.setFocusPainted(false);
    rb.setAlignmentX(Component.LEFT_ALIGNMENT);
}

ButtonGroup plans = new ButtonGroup();
for (JRadioButton rb : options) plans.add(rb);
fullOption.setSelected(true);

JPanel planPanel = new JPanel();
planPanel.setLayout(new BoxLayout(planPanel, BoxLayout.Y_AXIS));
planPanel.setOpaque(false);
planPanel.setAlignmentX(Component.LEFT_ALIGNMENT);
planPanel.setBorder(BorderFactory.createTitledBorder(
        "Select Payment Plan (choose one)"));
for (JRadioButton rb : options) {
    planPanel.add(rb);
    planPanel.add(Box.createVerticalStrut(2));
}

JComboBox<String> methodBox = new JComboBox<>(PAYMENT_METHODS);
methodBox.setFont(new Font("Segoe UI", Font.PLAIN, 12));
methodBox.setPreferredSize(new Dimension(320, 32));
methodBox.setMaximumSize(new Dimension(320, 32));

JTextField reference = new JTextField(generateReferenceNo());
reference.setEditable(false);
reference.setBackground(new Color(0xF0, 0xF0, 0xF0));
```

```
reference.setForeground(GRAY);
styleInput(reference);

JLabel balanceLabel = new JLabel(
        "Current outstanding balance: " + tenantBalance());
balanceLabel.setFont(new Font("Segoe UI", Font.BOLD, 11));
balanceLabel.setForeground(GRAY);
balanceLabel.setAlignmentX(Component.LEFT_ALIGNMENT);

JPanel fields = new JPanel(new GridBagLayout());
fields.setOpaque(false);
fields.setAlignmentX(Component.LEFT_ALIGNMENT);

GridBagConstraints gc = new GridBagConstraints();
gc.insets = new Insets(6, 4, 6, 4);
gc.fill = GridBagConstraints.HORIZONTAL;

gc.gridx = 0; gc.gridy = 0; gc.weightx = 0;
fields.add(label("Payment Method:", true), gc);
gc.gridx = 1; gc.weightx = 1;
fields.add(methodBox, gc);

gc.gridx = 0; gc.gridy = 1; gc.weightx = 0;
fields.add(label("Reference No.:", true), gc);
gc.gridx = 1; gc.weightx = 1;
fields.add(reference, gc);

JPanel wrapper = new JPanel();
wrapper.setLayout(new BoxLayout(wrapper, BoxLayout.Y_AXIS));
```

```
wrapper.setBorder(new EmptyBorder(6, 6, 6, 6));
wrapper.add(planPanel);
wrapper.add(Box.createVerticalStrut(10));
wrapper.add(balanceLabel);
wrapper.add(Box.createVerticalStrut(6));
wrapper.add(fields);

int result = JOptionPane.showConfirmDialog(this, wrapper,
        "Submit Payment", JOptionPane.OK_CANCEL_OPTION,
        JOptionPane.PLAIN_MESSAGE);
if (result != JOptionPane.OK_OPTION) return;

double value;
String description;
if (fullOption.isSelected()) {
    value = FULL_PAYMENT_AMOUNT;
    description = "Full Payment";
} else if (partialOption.isSelected()) {
    value = PARTIAL_PAYMENT_AMOUNT;
    description = "Partial Payment";
} else if (minimalOption.isSelected()) {
    value = MINIMAL_PAYMENT_AMOUNT;
    description = "Minimal Payment";
} else {
    value = ADVANCE_3MO_AMOUNT;
    description = "Advance 3 Months Payment";
}

String methodValue = methodBox.getSelectedItem().toString();
```

```
String referenceValue = reference.getText().trim();

try {
    int roomId = tenantRoomId();

    String sql = "INSERT INTO payments "
            + "(user_id,room_id,payment_date,description,amount,method,"
            + "status,reference_no) "
            + "VALUES(?,?,CURDATE(),?,?,?,'Pending',?)";

    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, currentUserId);
        if (roomId == 0) ps.setNull(2, Types.INTEGER);
        else ps.setInt(2, roomId);
        ps.setString(3, description);
        ps.setDouble(4, value);
        ps.setString(5, methodValue);
        ps.setString(6, referenceValue);
        ps.executeUpdate();
    }

    JOptionPane.showMessageDialog(this,
            String.format("%s of \u20B1%,.2f submitted for admin review.%n"
                    + "Reference No.: %s", description, value, referenceValue));
    showTenant();
    cardLayout.show(content, "Payments");

} catch (SQLException e) {
    showError(e);
```

```
        }
    }

    private void cancelPayment(JTable table, DefaultTableModel model) {
        int row = table.getSelectedRow();
        if (row < 0) {
            JOptionPane.showMessageDialog(this, "Select a payment first.");
            return;
        }

        int id = Integer.parseInt(model.getValueAt(row, 0).toString());
        String status = model.getValueAt(row, 5).toString();

        if (!"Pending".equalsIgnoreCase(status)) {
            JOptionPane.showMessageDialog(this,
                    "Only pending payments can be cancelled.");
            return;
        }

        if (!confirm("Cancel this pending payment?")) return;

        try (PreparedStatement ps = connection.prepareStatement(
                "DELETE FROM payments WHERE payment_id=? AND user_id=? AND
status='Pending'")) {
            ps.setInt(1, id);
            ps.setInt(2, currentUserId);
            ps.executeUpdate();
            showTenant();
            cardLayout.show(content, "Payments");
```

```
    } catch (SQLException e) {
        showError(e);
    }
}

private JPanel tenantApplication() {
    JPanel p = basePanel("Room Allocation & Applications",
            "View room availability and submit or cancel requests.");

    JPanel main = new JPanel(new BorderLayout(0, 15));
    main.setOpaque(false);

    JPanel currentRoomBanner = whiteCard();
    currentRoomBanner.setLayout(new BorderLayout(0, 5));
    currentRoomBanner.setMaximumSize(new Dimension(Integer.MAX_VALUE, 80));
    currentRoomBanner.setPreferredSize(new Dimension(0, 80));

    JLabel bannerTitle = new JLabel("Current Assigned Room");
    bannerTitle.setFont(new Font("Segoe UI", Font.BOLD, 11));
    bannerTitle.setForeground(GRAY);

    JLabel bannerValue = new JLabel(tenantRoom());
    bannerValue.setFont(new Font("Segoe UI", Font.BOLD, 18));
    bannerValue.setForeground(color1);

    currentRoomBanner.add(bannerTitle, BorderLayout.NORTH);
    currentRoomBanner.add(bannerValue, BorderLayout.CENTER);

    main.add(currentRoomBanner, BorderLayout.NORTH);
```

```
String[] cols = {"Room", "Capacity", "Occupied", "Available",
        "Monthly Rent", "Status"};
DefaultTableModel roomsModel = new DefaultTableModel(cols, 0) {
    @Override
    public boolean isCellEditable(int r, int c) { return false; }
};

try (PreparedStatement ps = connection.prepareStatement(
        "SELECT room_number,capacity,occupied,monthly_rent "
        + "FROM rooms ORDER BY room_id");
     ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        int occ = rs.getInt("occupied");
        int cap = rs.getInt("capacity");
        roomsModel.addRow(new Object[]{
                rs.getString("room_number"),
                cap,
                occ,
                cap - occ,
                "\u20B1" + rs.getBigDecimal("monthly_rent"),
                occ < cap ? "AVAILABLE" : "FULL"
        });
    }
} catch (SQLException e) {
    showError(e);
}

JTable roomsTable = new JTable(roomsModel);
```

```
    styleTable(roomsTable);
    roomsTable.getColumnModel().getColumn(5).setCellRenderer(new StatusRenderer());

    JPanel buttons = new JPanel(new FlowLayout(FlowLayout.RIGHT));
    JButton apply = button("Apply for Selected Room", color1);
    JButton cancel = button("Cancel My Pending Application", RED);

    apply.addActionListener(e -> applyForSelectedRoom(roomsTable));
    cancel.addActionListener(e -> cancelApplication());

    buttons.add(apply);
    buttons.add(cancel);

    main.add(new JScrollPane(roomsTable), BorderLayout.CENTER);
    main.add(buttons, BorderLayout.SOUTH);

    p.add(main, BorderLayout.CENTER);
    return p;
}

private void applyForSelectedRoom(JTable table) {
    int row = table.getSelectedRow();
    if (row < 0) {
        JOptionPane.showMessageDialog(this,
                "Select an available room first.");
        return;
    }

    int available = Integer.parseInt(table.getValueAt(row, 3).toString());
```

```
if (available <= 0) {
    JOptionPane.showMessageDialog(this, "That room is full.");
    return;
}

String roomNumber = table.getValueAt(row, 0).toString();

if (hasPendingApplication()) {
    JOptionPane.showMessageDialog(this,
            "You already have a pending room application.");
    return;
}

JTextField date = new JTextField(LocalDate.now().toString());
JTextArea message = new JTextArea(4, 25);
message.setLineWrap(true);
message.setWrapStyleWord(true);

Object[] fields = {
        "Selected Room:", roomNumber,
        "Preferred Move-in Date (YYYY-MM-DD):", date,
        "Message (optional):", new JScrollPane(message)
};

int result = JOptionPane.showConfirmDialog(this, fields,
        "Room Application", JOptionPane.OK_CANCEL_OPTION);
if (result != JOptionPane.OK_OPTION) return;

try {
```

```
java.sql.Date moveIn = java.sql.Date.valueOf(date.getText().trim());

String sql = "SELECT room_id FROM rooms "
        + "WHERE room_number=? AND occupied<capacity";
int roomId = 0;

try (PreparedStatement ps = connection.prepareStatement(sql)) {
    ps.setString(1, roomNumber);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) roomId = rs.getInt("room_id");
    }
}

if (roomId == 0) {
    JOptionPane.showMessageDialog(this,
            "The room is no longer available.");
    return;
}

String insert = "INSERT INTO applications "
        + "(user_id,room_id,move_in_date,status,message) "
        + "VALUES(?,?,?,'Pending',?)";

try (PreparedStatement ps = connection.prepareStatement(insert)) {
    ps.setInt(1, currentUserId);
    ps.setInt(2, roomId);
    ps.setDate(3, moveIn);
    ps.setString(4, message.getText().trim());
    ps.executeUpdate();
```

```
        }

        JOptionPane.showMessageDialog(this,
                "Room application submitted.");
    } catch (IllegalArgumentException e) {
        JOptionPane.showMessageDialog(this,
                "Use a valid date: YYYY-MM-DD.");
    } catch (SQLException e) {
        showError(e);
    }
}

private void cancelApplication() {
    String sql = "SELECT application_id FROM applications "
            + "WHERE user_id=? AND status='Pending' "
            + "ORDER BY application_id DESC LIMIT 1";

    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, currentUserId);
        try (ResultSet rs = ps.executeQuery()) {
            if (!rs.next()) {
                JOptionPane.showMessageDialog(this,
                        "You have no pending application.");
                return;
            }

            int id = rs.getInt(1);

            if (!confirm("Cancel your pending room application?")) return;
```

```
            try (PreparedStatement del = connection.prepareStatement(
                    "UPDATE applications SET status='Cancelled' "
                    + "WHERE application_id=? AND user_id=?")) {
                del.setInt(1, id);
                del.setInt(2, currentUserId);
                del.executeUpdate();
            }

            JOptionPane.showMessageDialog(this, "Application cancelled.");
            showTenant();
            cardLayout.show(content, "Application");
        }
    } catch (SQLException e) {
        showError(e);
    }
}

private JComboBox<RoomItem> roomCombo(boolean includeNone) {
    JComboBox<RoomItem> combo = new JComboBox<>();

    if (includeNone) combo.addItem(new RoomItem(0, "No Room"));

    String sql = "SELECT room_id, room_number, capacity, occupied "
            + "FROM rooms ORDER BY room_number";

    try (PreparedStatement ps = connection.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
```

```
            if (rs.getInt("occupied") < rs.getInt("capacity") || !includeNone) {
                combo.addItem(new RoomItem(
                        rs.getInt("room_id"),
                        "Room " + rs.getString("room_number")
                                + " (" + rs.getInt("occupied")
                                + "/" + rs.getInt("capacity") + ")"));
            }
        }
    } catch (SQLException e) {
        showError(e);
    }

    return combo;
}

private boolean containsRoom(JComboBox<RoomItem> combo, int roomId) {
    for (int i = 0; i < combo.getItemCount(); i++) {
        if (combo.getItemAt(i).id == roomId) return true;
    }
    return false;
}

private void selectRoom(JComboBox<RoomItem> combo, int roomId) {
    for (int i = 0; i < combo.getItemCount(); i++) {
        if (combo.getItemAt(i).id == roomId) {
            combo.setSelectedIndex(i);
            return;
        }
    }
```

```
    combo.setSelectedIndex(0);
}

private void changeRoomOccupancy(int roomId, int change) throws SQLException {
    String sql = "UPDATE rooms SET occupied=GREATEST(0, occupied+?) "
            + "WHERE room_id=? AND occupied+? <= capacity";

    if (change < 0) {
        try (PreparedStatement ps = connection.prepareStatement(
                "UPDATE rooms SET occupied=GREATEST(0,occupied+?) "
                + "WHERE room_id=?")) {
            ps.setInt(1, change);
            ps.setInt(2, roomId);
            ps.executeUpdate();
        }
    } else {
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setInt(1, change);
            ps.setInt(2, roomId);
            ps.setInt(3, change);
            ps.executeUpdate();
        }
    }
}

private boolean canAddToRoom(int roomId) throws SQLException {
    String sql = "SELECT occupied, capacity FROM rooms WHERE room_id=?";
    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, roomId);
```

```
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next())
                return rs.getInt("occupied") < rs.getInt("capacity");
        }
    }
    return false;
}

private boolean hasPendingApplication() {
    String sql = "SELECT COUNT(*) FROM applications "
            + "WHERE user_id=? AND status='Pending'";
    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, currentUserId);
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next() && rs.getInt(1) > 0;
        }
    } catch (SQLException e) {
        showError(e);
    }
    return false;
}

/**
 * Table name is validated against a whitelist so it can never be
 * influenced from outside this class.
 */
private String count(String table, String condition) {
    if (table == null || !COUNTABLE_TABLES.contains(table.toLowerCase())) {
        return "0";
```

```
    }

    String sql = "SELECT COUNT(*) FROM " + table
            + (condition == null ? "" : " WHERE " + condition);

    try (PreparedStatement ps = connection.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {
        if (rs.next()) return String.valueOf(rs.getInt(1));
    } catch (SQLException e) {
        showError(e);
    }
    return "0";
}

private String availableSlots() {
    String sql = "SELECT COALESCE(SUM(capacity-occupied),0) FROM rooms";
    try (PreparedStatement ps = connection.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {
        if (rs.next()) return String.valueOf(rs.getInt(1));
    } catch (SQLException e) {
        showError(e);
    }
    return "0";
}

private String occupancyRate() {
    String sql = "SELECT COALESCE(SUM(occupied),0), COALESCE(SUM(capacity),0) "
            + "FROM rooms";
    try (PreparedStatement ps = connection.prepareStatement(sql);
```

```
             ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                int occ = rs.getInt(1);
                int cap = rs.getInt(2);
                if (cap == 0) return "0%";
                return Math.round(occ * 100.0 / cap) + "%";
            }
        } catch (SQLException e) {
            showError(e);
        }
        return "0%";
    }

    private String monthlyRevenue() {
        String sql = "SELECT COALESCE(SUM(amount),0) FROM payments "
                + "WHERE status='Paid' AND MONTH(payment_date)=MONTH(CURDATE()) "
                + "AND YEAR(payment_date)=YEAR(CURDATE())";
        try (PreparedStatement ps = connection.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return "\u20B1" + rs.getBigDecimal(1);
        } catch (SQLException e) {
            showError(e);
        }
        return "\u20B10.00";
    }

    private String totalPaidAmount() {
        String sql = "SELECT COALESCE(SUM(amount),0) FROM payments WHERE
status='Paid'";
```

```
        try (PreparedStatement ps = connection.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return "\u20B1" + rs.getBigDecimal(1);
        } catch (SQLException e) {
            showError(e);
        }
        return "\u20B10.00";
    }

    private String totalPendingAmount() {
        String sql = "SELECT COALESCE(SUM(amount),0) FROM payments WHERE
status='Pending'";
        try (PreparedStatement ps = connection.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return "\u20B1" + rs.getBigDecimal(1);
        } catch (SQLException e) {
            showError(e);
        }
        return "\u20B10.00";
    }

    private String projectedMonthly() {
        String sql = "SELECT COALESCE(SUM(monthly_rent * occupied),0) FROM rooms";
        try (PreparedStatement ps = connection.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return "\u20B1" + rs.getBigDecimal(1);
        } catch (SQLException e) {
            showError(e);
        }
```

```
    return "\u20B10.00";
}

private int tenantRoomId() {
    String sql = "SELECT room_id FROM users WHERE user_id=?";
    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, currentUserId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                int id = rs.getInt(1);
                return rs.wasNull() ? 0 : id;
            }
        }
    } catch (SQLException e) {
        showError(e);
    }
    return 0;
}

private String tenantRoom() {
    String sql = "SELECT r.room_number FROM users u "
            + "LEFT JOIN rooms r ON u.room_id=r.room_id WHERE u.user_id=?";

    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, currentUserId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next() && rs.getString(1) != null)
                return "Room " + rs.getString(1);
        }
```

```
    } catch (SQLException e) {
        showError(e);
    }
    return "Not Assigned";
}

private String tenantMonthlyRent() {
    String sql = "SELECT r.monthly_rent FROM users u "
            + "LEFT JOIN rooms r ON u.room_id=r.room_id WHERE u.user_id=?";
    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, currentUserId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next() && rs.getBigDecimal(1) != null)
                return "\u20B1" + rs.getBigDecimal(1);
        }
    } catch (SQLException e) {
        showError(e);
    }
    return "\u20B10.00";
}

private String tenantBalance() {
    String sql = "SELECT COALESCE(SUM(amount),0) FROM payments "
            + "WHERE user_id=? AND status IN ('Pending','Overdue')";

    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, currentUserId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return "\u20B1" + rs.getBigDecimal(1);
```

```
        }
    } catch (SQLException e) {
        showError(e);
    }
    return "\u20B10.00";
}

private String tenantTotalPaid() {
    String sql = "SELECT COALESCE(SUM(amount),0) FROM payments "
            + "WHERE user_id=? AND status='Paid'";
    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, currentUserId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return "\u20B1" + rs.getBigDecimal(1);
        }
    } catch (SQLException e) {
        showError(e);
    }
    return "\u20B10.00";
}

private double tenantTotalPaidValue() {
    String sql = "SELECT COALESCE(SUM(amount),0) FROM payments "
            + "WHERE user_id=? AND status='Paid'";
    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, currentUserId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return rs.getDouble(1);
        }
```

```
    } catch (SQLException e) {
        showError(e);
    }
    return 0;
}

private double tenantTotalBilled() {
    String sql = "SELECT COALESCE(SUM(amount),0) FROM payments WHERE user_id=?";
    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, currentUserId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return rs.getDouble(1);
        }
    } catch (SQLException e) {
        showError(e);
    }
    return 0;
}

private String tenantStatus() {
    try (PreparedStatement ps = connection.prepareStatement(
            "SELECT status FROM users WHERE user_id=?")) {
        ps.setInt(1, currentUserId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return rs.getString(1);
        }
    } catch (SQLException e) {
        showError(e);
    }
```

```
    return "Unknown";
}

private String nextDue() {
    String sql = "SELECT MIN(payment_date) FROM payments "
            + "WHERE user_id=? AND status IN ('Pending','Overdue')";

    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setInt(1, currentUserId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next() && rs.getDate(1) != null)
                return rs.getDate(1).toString();
        }
    } catch (SQLException e) {
        showError(e);
    }
    return "None";
}

private void refreshAdmin(String page) {
    showAdmin();
    cardLayout.show(content, page);
}

private static class StatusRenderer extends DefaultTableCellRenderer {
    private static final long serialVersionUID = 1L;
    @Override
    public Component getTableCellRendererComponent(JTable table, Object value,
            boolean isSelected, boolean hasFocus, int row, int column) {
```

```
        Component c = super.getTableCellRendererComponent(table, value,
                isSelected, hasFocus, row, column);

        if (value != null) {
            String v = value.toString().toUpperCase();
            setHorizontalAlignment(SwingConstants.CENTER);
            setFont(new Font("Segoe UI", Font.BOLD, 11));

            if (!isSelected) {
                if ("PAID".equals(v) || "ACTIVE".equals(v)
                        || "APPROVED".equals(v) || "AVAILABLE".equals(v)) {
                    c.setForeground(new Color(0x10, 0xB9, 0x81));
                } else if ("PENDING".equals(v) || "INACTIVE".equals(v)) {
                    c.setForeground(new Color(0xF5, 0x9E, 0x0B));
                } else if ("REJECTED".equals(v) || "FULL".equals(v)
                        || "CANCELLED".equals(v)) {
                    c.setForeground(new Color(0xEF, 0x44, 0x44));
                } else {
                    c.setForeground(new Color(0x1F, 0x29, 0x37));
                }
            }
        }
        return c;
    }
}

private static class RoomItem {
    int id;
    String name;
```

```
        RoomItem(int id, String name) {
            this.id = id;
            this.name = name;
        }

        @Override
        public String toString() {
            return name;
        }
}
public static void main(String[] args) {
SwingUtilities.invokeLater(() ->
new Ocamporesidenc().setVisible(true));
}
}
```